



Co-occurrence order-preserving pattern mining with keypoint alignment for time series

YOUXI WU, Hebei University of Technology, Tianjin, China

ZHEN WANG, Hebei University of Technology, Tianjin, China

YAN LI*, Hebei University of Technology, Tianjin, China

YINGCHUN GUO, Hebei University of Technology, Tianjin, China

HE JIANG, Dalian University of Technology, Dalian, China

XINGQUAN ZHU, Florida Atlantic University, FL, USA

XINDONG WU, Hefei University of Technology, Hefei, China

Recently, order-preserving pattern (OPP) mining has been proposed to discover some patterns, which can be seen as trend changes in time series. Although existing OPP mining algorithms have achieved satisfactory performance, they discover all frequent patterns. However, in some cases, users focus on a particular trend and its associated trends. To efficiently discover trend information related to a specific prefix pattern, this paper addresses the issue of co-occurrence OPP mining (COP) and proposes an algorithm named COP-Miner to discover COPs from historical time series. COP-Miner consists of three parts: extracting keypoints, preparation stage, and iteratively calculating supports and mining frequent COPs. Extracting keypoints is used to obtain local extreme points of patterns and time series. The preparation stage is designed to prepare for the first round of mining, which contains four steps: obtaining the suffix OPP of the keypoint sub-time series, calculating the occurrences of the suffix OPP, verifying the occurrences of the keypoint sub-time series, and calculating the occurrences of all fusion patterns of the keypoint sub-time series. To further improve the efficiency of support calculation, we propose a support calculation method with an ending strategy that uses the occurrences of prefix and suffix patterns to calculate the occurrences of superpatterns. Experimental results indicate that COP-Miner outperforms the other competing algorithms in running time and scalability. Moreover, COPs with keypoint alignment yield better prediction performance.

CCS Concepts: • **Theory of computation** → **Design and analysis of algorithms**; • **Computing methodologies** → **Knowledge representation and reasoning**.

Additional Key Words and Phrases: pattern mining, time series, keypoint alignment, order-preserving, co-occurrence pattern

*Corresponding authors.

Authors' addresses: Youxi Wu, School of Artificial Intelligence, Hebei University of Technology, Tianjin, China, 300401, wuc567@163.com; Zhen Wang, School of Artificial Intelligence, Hebei University of Technology, Tianjin, China, 300401, 17302297683@163.com; Yan Li, School of Economics and Management, Hebei University of Technology, Tianjin, China, 300401, lywuc@163.com; Yingchun Guo, School of Artificial Intelligence, Hebei University of Technology, Tianjin, China, 300401, gyc@scse.hebut.edu.cn; He Jiang, School of Software, Dalian University of Technology, Dalian, China, 116023, jianghe@dlut.edu.cn; Xingquan Zhu, Department of Computer & Electrical Engineering and Computer Science, Florida Atlantic University, FL, USA, 33431, xzhu3@fau.edu; Xindong Wu, Key Laboratory of Knowledge Engineering with Big Data (the Ministry of Education of China), Hefei University of Technology, Hefei, China, 230009, xwu@hfut.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 2158-656X/2024/4-ART

<https://doi.org/10.1145/3658450>

1 INTRODUCTION

A time series is a set of numerical values derived from consecutive records over a fixed time interval. Time series exist in many fields, such as market prediction [63], taxi demand prediction [40], stock price prediction [25, 28], and brain EEG analysis [7]. Mining hidden information in time series can be used for big data analysis techniques [48], such as clustering [16], classification [35], pattern recognition [31, 62] and predication [23]. Therefore, time series mining methods are significant and various algorithms have been proposed [12, 13]. For example, inspired by the three-way decisions theory [50, 64], the NTP-Miner algorithm converts time series to characters and divides the alphabet into three regions: strong, medium, and weak, which can be applied to cluster analysis [53]. The TBOPE classification algorithm builds an ensemble classifier that can improve classification performance [3]. The OWSP-Miner algorithm can recognize self-adaptive one-off weak-gap strong patterns, which are more valuable in real life [57]. Most time series mining must use representation methods to convert time series into discretized characters [46]. Commonly used methods are piecewise linear approximation (PLA) [41], piecewise aggregate approximation (PAA) [5], and symbolic aggregate approximation (SAX) [32]. However, the above-mentioned symbolization methods easily lead to the loss of trend information inside the original time series.

Recently, a method without symbolization named order-preserving pattern (OPP) matching has been proposed which can find all occurrences of a given OPP on numeric strings [20]. An OPP can represent a trend, which is a relative order that can effectively reflect fluctuations without discretizing the values into symbols [54]. Inspired by OPP matching, a new sequential pattern mining method, named OPP mining was proposed [52], which can discover frequent OPPs in time series. To improve the mining performance and discover all strong rules, order-preserving rule mining was explored [59]. However, there are two main issues in the current research.

(1) The existing algorithms cannot cope with distortion and mine redundant patterns, since these algorithms mine directly on the original time series. For example, an OPP is (1,2,3,4,5), which represents an uptrend and can be replaced by the already discovered pattern (1,2), since (1,2) also represents an uptrend. Therefore, it is necessary to discover fewer and more targeted patterns that are more meaningful and unaffected by distortion.

(2) More importantly, the existing algorithms, such as OPP-Miner [52] and OPR-Miner [59], mine all of the patterns and use them for time series clustering and classification. But, in some cases, researchers want to discover patterns related to a specific pattern in historical data, and then utilize a summary of past trends to predict future changes. In this case, mining all the patterns would yield numerous meaningless results.

To tackle the above two issues, two methods need to be employed, respectively. For distortion, the keypoint alignment method [22] can be used. Thus, to reduce distortion interference in time series, we do not mine on the original time series, but rather on the time series after extracting keypoints. Moreover, to avoid mining redundant patterns, co-occurrence pattern mining can be selected [15, 29]. The reason is as follows. Although top- k pattern mining [8, 11], closed pattern mining [42, 60], and maximal pattern mining [30, 43] can reduce the number of patterns, users may be interested in a part of patterns, rather than all patterns, since all patterns are not sufficiently targeted. For example, when users know a specific prefix pattern in advance to make predictions, they only want to discover patterns with the same specific prefix pattern, and this mining method is called co-occurrence pattern mining [15, 29]. This problem still exists in time series. For example, in time series prediction, users hope to use historical data to predict future values. Therefore, users know the latest values that can be further extracted as a prefix pattern. Based on the prefix pattern, users can predict future values. Thus, co-occurrence pattern mining is more targeted. An illustrative example is as follows.

EXAMPLE 1. Suppose we have a time series $t = (16, 8, 11, 10, 12, 16, 17, 13, 20, 18, 21, 22, 18, 14, 21, 24, 23, 27, 25)$ shown in Fig. 1 and a predefined support threshold $\text{minsup}=3$. According to OPP-Miner, there are seven frequent OPPs: $\{(1,2), (2,1), (1,2,3), (1,3,2), (2,1,3), (1,3,2,4), (2,1,3,4)\}$. Obviously, it is difficult for users to apply these trends information directly. We know that the latest three values of t are $(t_{17}, t_{18}, t_{19}) = (23, 27, 25)$ whose relative order is (1,3,2), since

$t_{17}=23$ is the smallest value, $t_{18}=27$ is the third smallest value, and $t_{19}=25$ is the second smallest value. Moreover, it is easy to know OPP $(1,3,2)$ occurs four times in $\mathbf{t}$: (t_2, t_3, t_4) , (t_8, t_9, t_{10}) , (t_{15}, t_{16}, t_{17}) , and (t_{17}, t_{18}, t_{19}) . Thus, $(1,3,2)$ is a frequent OPP. Hence, when we make a prediction, actually, we know the specific prefix pattern.

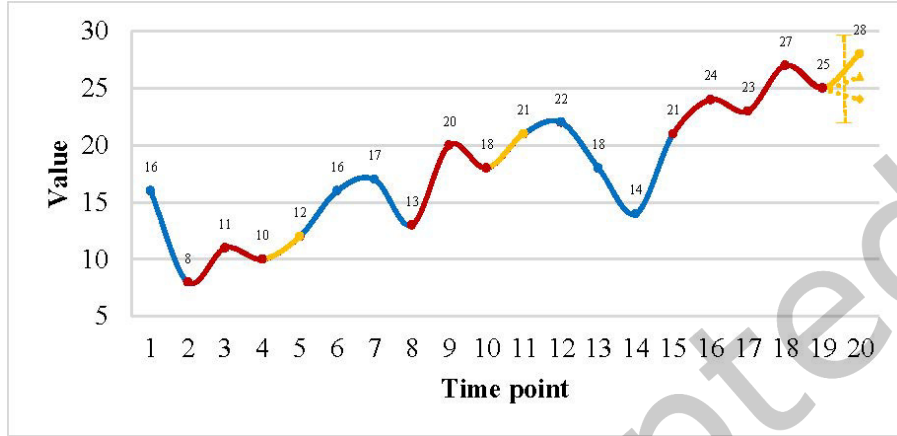


Fig. 1. Time series $\mathbf{t}$. The relative order of sub-time series $(t_2, t_3, t_4) = (8, 11, 10)$ is $(1, 3, 2)$, since $t_2=8$ is the smallest value, $t_3=11$ is the third smallest value, and $t_4=10$ is the second smallest value. Similarly, the relative orders of sub-time series (t_8, t_9, t_{10}) , (t_{15}, t_{16}, t_{17}) , and (t_{17}, t_{18}, t_{19}) are also $(1, 3, 2)$. Assuming that we know the values t_1 to t_{19} , we can discover the potential trends, and apply these trends and the recent values, such as (t_{17}, t_{18}, t_{19}) to predict the range of t_{20} .

Now, we use the specific prefix pattern $(1, 3, 2)$ to predict the further trend. Therefore, we only need to discover the patterns whose prefix pattern is $(1, 3, 2)$. These discovered patterns are called co-occurrence order-preserving patterns (COPs). According to historical information, we know that $(1, 3, 2, 4)$ is also a frequent OPP and its prefix pattern is $(1, 3, 2)$. Hence, $(1, 3, 2, 4)$ is a COP. Thus, compared with seven frequent OPPs, COP $(1, 3, 2, 4)$ is more targeted in predicting future trends. For example, t_{20} is 28, which is greater than t_{18} . Thus, the relative order of $(t_{17}, t_{18}, t_{19}, t_{20})$ is $(1, 3, 2, 4)$, which means that the prediction is correct. If t_{20} is less than t_{18} , then the prediction is not correct.

From the above description, mining COPs with keypoint alignment for time series prediction is meaningful. The main contributions are as follows.

- To avoid mining irrelevant trends and to obtain better prediction performance for time series, we explore COP mining, which can mine all COPs with the same prefix pattern, and we propose the COP-Miner algorithm.
- COP-Miner is composed of three parts: extracting keypoints to reduce distortion interference and avoid mining redundant patterns, preparation stage to prepare for the first round of mining, and iteratively calculating supports of superpatterns and mining frequent COPs.
- We propose the concept of fusion pattern which can effectively reduce the number of candidate patterns.
- To further improve the efficiency of support calculation, we propose a support calculation method with an ending strategy which uses the occurrences of prefix and suffix patterns to calculate the occurrences of superpatterns.
- To validate the performance of COP-Miner, we select ten competing algorithms and nine real time series datasets. Our experimental results verify that COP-Miner outperforms other competing algorithms and COPs with keypoint alignment yield better prediction performance.

This paper is organized as follows. Section 2 gives an overview of related work. Section 3 defines the problem. Section 4 proposes the COP-Miner algorithm. Section 5 reports the performance of COP-Miner. Section 6 concludes this paper.

2 RELATED WORK

Sequential pattern mining (SPM) is an important research direction in big data analysis [9, 36, 47, 56], which is widely used in bioinformatics [65], keyword extraction [14, 61], feature selection [33, 34, 58], and sequence classification [17, 24, 38]. To meet different needs in real life, various pattern mining methods have been proposed [10], such as episode pattern mining [2], negative SPM [49], three-way SPM [6, 37], contrast SPM [26, 54], high-utility SPM [18, 39, 51], spatial co-location pattern mining [44, 45], and gap constraints SPM [55]. However, most of these schemes aim to discover all patterns, which leads to numerous information that is not of interest to users. To reduce the number of mined patterns, top- k SPM [8, 11], closed SPM [42, 60], and maximal SPM [30, 43] have been proposed. However, in some cases, users have certain prior knowledge and want to obtain patterns with a specific prefix pattern. To solve this problem, some co-occurrence SPM methods were proposed, such as maximal co-occurrence SPM [29] and top- k co-occurrence patterns mining [1].

However, the above methods mainly focused on mining character sequences (such as biological and customer purchase behavior sequences), which are difficult to apply directly to time series (such as temperature changes and stock trading volumes). At present, the prevailing solution is to discretize the values into symbols by symbolic methods, such as PLA [41], PAA [5], and SAX [32], and then apply the above SPM algorithms to find frequent patterns in time series. But the drawbacks of symbolization cannot be ignored. First of all, the symbolization of time series may bring some noise. More importantly, it is difficult to discover the potential trends in time series, since the symbolization methods convert the real values into different symbols which means that these methods focus too much on specific values and ignore trends.

We have noticed a new approach in the field of pattern matching named OPP matching, which does not require symbolization of time series. For example, Kim et al. [20] proposed an OPP matching solution and indicated trends based on the relative order of numerical values. Moreover, Kim et al. [21] studied an approximate order-preserving pattern matching problem with scaling. Inspired by OPP matching, our previous work proposed the OPP-Miner algorithm to discover all frequent OPPs [52] and further proposed the OPR-Miner algorithm to mine strong order-preserving rules from all frequent OPPs [59]. The common characteristic of OPP-Miner and OPR-Miner is the discovery of all frequent patterns.

To mine patterns with user-specified prefix pattern, inspired by co-occurrence pattern mining [15, 29], we present a COP mining scheme for time series prediction. COP mining can discover fewer and more targeted patterns. Although we can employ OPP-Miner [52] or EFO-Miner [59] to discover all frequent OPPs at first, and then select the COPs, it is obviously inefficient and belongs to the brute-force approach, since this approach will discover many irrelevant patterns. Therefore, to discover COPs efficiently, we specially design an algorithm named COP-Miner.

3 PROBLEM DEFINITION

This section presents the related concepts of COPs with specific prefix patterns for time series.

DEFINITION 1. (*Original time series*) An original time series is a finite sequence of numerical values derived from consecutive records over a fixed time interval, denoted by $\mathbf{t} = (t_1, \dots, t_i, \dots, t_N)$, where $1 \leq i \leq N$.

Considering that a time series can be disturbed easily by distortion, resulting in warping, scaling, and data perturbation, we extract the keypoints of a time series to eliminate distortion.

DEFINITION 2. (Keypoint time series) For a time series $\mathbf{t}$, t_i is a keypoint, iff it is the local minimum or local maximum and satisfies the following conditions [22]. The new time series $\mathbf{k}$, called keypoint time series, is a time series composed of all keypoints in $\mathbf{t}$.

- (1) If $i=1$ or $i=N$, then t_i is the first or last point.
- (2) If $t_i < t_{i-1}$ and $t_i \leq t_{i+1}$, or $t_i \leq t_{i-1}$ and $t_i < t_{i+1}$, then t_i is a local minimum point.
- (3) If $t_i > t_{i-1}$ and $t_i \geq t_{i+1}$, or $t_i \geq t_{i-1}$ and $t_i > t_{i+1}$, then t_i is a local maximum point.

An illustrative example of time series and keypoint time series is shown as follows.

EXAMPLE 2. Suppose we have a time series $\mathbf{t} = (16, 8, 11, 10, 12, 16, 17, 13, 20, 18, 21, 22, 18, 14, 21, 24, 23, 27, 25, 28)$. $t_1=16$ is a keypoint, since it is the first point. $t_2=8$ is a keypoint, since $t_2=8 < t_1=16$ and $t_2=8 < t_3=11$. However, t_5 is not a keypoint, since $t_4=10 < t_5=12 < t_6=16$. After extracting all keypoints, $\mathbf{k} = (16, 8, 11, 10, 17, 13, 20, 18, 22, 14, 24, 23, 27, 25, 28)$.

DEFINITION 3. (Sub-time series and keypoint sub-time series) Sub-time series $\mathbf{p}$ is also a group of numerical values $(p_1, \dots, p_j, \dots, p_M)$ ($1 \leq j \leq M$). After extracting keypoints, $\mathbf{q} = (q_1, \dots, q_j, \dots, q_m)$ ($1 \leq j \leq m$) is a keypoint sub-time series of $\mathbf{p}$.

DEFINITION 4. (OPP) The rank of an element q_j in a sub-time series $\mathbf{q}$ is its rank order, denoted by $o_{\mathbf{q}}(q_j)$. An order-preserving pattern (OPP) is represented by the relative order of $\mathbf{q}$, described by $\mathbf{o} = R(\mathbf{q}) = (o_{\mathbf{q}}(q_1), o_{\mathbf{q}}(q_2), \dots, o_{\mathbf{q}}(q_m))$.

EXAMPLE 3. Suppose we have a sub-time series $\mathbf{p} = (5, 3, 7, 13, 8)$. After extracting keypoints of $\mathbf{p}$, the new keypoints sub-time series $\mathbf{q}$ is $(5, 3, 13, 8)$. Since 5 is the second smallest value in $\mathbf{q}$, its rank order is two, i.e., $o_{\mathbf{q}}(5)=2$. Similarly, $o_{\mathbf{q}}(3)=1$, $o_{\mathbf{q}}(13)=4$, and $o_{\mathbf{q}}(8)=3$. The OPP of $\mathbf{q}$ is $R(\mathbf{q}) = (2, 1, 4, 3)$.

Now, we give the definitions of occurrence, support, and frequent OPP.

DEFINITION 5. (Occurrence and support) For a keypoint time series $\mathbf{k} = (k_1, \dots, k_i, \dots, k_n)$ and an OPP $\mathbf{o} = (o_1, \dots, o_j, \dots, o_m)$, $\langle i \rangle$ is an occurrence of $\mathbf{o}$ in $\mathbf{k}$, iff $R(\mathbf{k}') = \mathbf{o}$, where $\mathbf{k}'$ is a sub-time series $(k_{i-m+1}, k_{i-m+2}, \dots, k_i)$ ($1 \leq i-m+1$ and $i \leq n$). The number of occurrences of $\mathbf{o}$ in $\mathbf{k}$ is called the support, denoted by $\text{sup}(\mathbf{o}, \mathbf{k})$.

DEFINITION 6. (Frequent OPP) If the support of OPP $\mathbf{o}$ in $\mathbf{k}$ is no less than the user-specified support threshold minsup (i.e., $\text{sup}(\mathbf{o}, \mathbf{k}) \geq \text{minsup}$), then $\mathbf{o}$ is a frequent OPP.

EXAMPLE 4. Suppose we have the same keypoint time series $\mathbf{k}$ as in Example 2 and keypoint sub-time series $\mathbf{q} = (5, 3, 13, 8)$, and we set $\text{minsup}=3$. According to Definition 4, $\mathbf{o} = R(\mathbf{q}) = (2, 1, 4, 3)$. The relative order of sub-time series $(k_3, k_4, k_5, k_6) = (11, 10, 17, 13)$ are $(2, 1, 4, 3)$. Similarly, $P(k_5, k_6, k_7, k_8) = P(k_9, k_{10}, k_{11}, k_{12}) = P(k_{11}, k_{12}, k_{13}, k_{14}) = (2, 1, 4, 3)$. Therefore, $\langle 6 \rangle$, $\langle 8 \rangle$, $\langle 12 \rangle$, and $\langle 14 \rangle$ are four occurrences of $\mathbf{o}$ in $\mathbf{k}$. Thus, $\text{sup}(\mathbf{o}, \mathbf{k}) = 4$ is no less than minsup . Hence, $\mathbf{o} = (2, 1, 4, 3)$ is a frequent OPP.

Based on the definition of frequent OPP, we present the definitions of prefix OPP, COP, and our problem.

DEFINITION 7. (Prefix OPP and suffix OPP, order-preserving subpattern and order-preserving super-pattern) For an OPP $\mathbf{x} = (x_1, x_2, \dots, x_m)$, pattern $\mathbf{e} = P(x_1, x_2, \dots, x_{m-1})$ is the prefix OPP of $\mathbf{x}$, denoted by $\mathbf{e} = \text{prefixorder}(\mathbf{x})$, and pattern $\mathbf{s} = P(x_2, x_3, \dots, x_m)$ is the suffix OPP of $\mathbf{x}$, denoted by $\mathbf{s} = \text{suffixorder}(\mathbf{x})$. Moreover, $\mathbf{e}$ and $\mathbf{s}$ are the order-preserving subpatterns of $\mathbf{x}$, and $\mathbf{x}$ is the order-preserving superpattern of $\mathbf{e}$ and $\mathbf{s}$.

DEFINITION 8. (COP) Suppose we have a frequent OPP $\mathbf{c} = (c_1, c_2, \dots, c_k)$. If the relative order of $(c_1, c_2, \dots, c_m)$ ($m < k$) is $\mathbf{o} = (o_1, o_2, \dots, o_m)$, then pattern $\mathbf{c}$ is a COP of $\mathbf{o}$.

DEFINITION 9. (Problem statement) Suppose we have a time series $\mathbf{t}$, a user-specified minimum support threshold minsup , and a sub-time series $\mathbf{p}$. After extracting keypoints, we obtain keypoint time series $\mathbf{k}$ and keypoint sub-time series $\mathbf{q}$. Then, we get OPP $\mathbf{o} = R(\mathbf{q})$. Our goal is to discover all frequent COPs of pattern $\mathbf{o}$ in $\mathbf{k}$.

EXAMPLE 5. We use the same keypoint time series $\mathbf{k}$ as in Example 2 and keypoint sub-time series $\mathbf{q} = (5, 3, 13, 8)$, and we set $\text{minsup} = 3$. According to Example 4, $R(\mathbf{q}) = (2, 1, 4, 3)$. According to Definition 5, we know that $\langle 7 \rangle$, $\langle 9 \rangle$, $\langle 13 \rangle$, and $\langle 15 \rangle$ are four occurrences of pattern $\mathbf{c} = (2, 1, 4, 3, 5)$. Thus, pattern $\mathbf{c}$ is a frequent COP of $\mathbf{q}$, since its prefix OPP is $(2, 1, 4, 3)$ which is the same as $R(\mathbf{q})$.

For clarification, the used symbols are listed in Table 1.

Table 1. Notation description

Symbol	Description
$\mathbf{t} = (t_1, \dots, t_i, \dots, t_N)$	Original time series
$\mathbf{k} = (k_1, \dots, k_i, \dots, k_n)$	Keypoint time series
$\mathbf{p} = (p_1, \dots, p_j, \dots, p_M)$	User-specified sub-time series
$\mathbf{q} = (q_1, \dots, q_j, \dots, q_m)$	Keypoint sub-time series
$\mathbf{o} = (o_1, \dots, o_j, \dots, o_m)$	A frequent OPP, $\mathbf{o} = R(\mathbf{q})$
$\text{Occ}_{\mathbf{o}} = \{go_1, \dots, go_i, \dots, go_y\}$	Occurrences of OPP $\mathbf{o}$
$\mathbf{e} = \text{prefixorder}(\mathbf{x})$	Prefix OPP of $\mathbf{x}$
$\mathbf{s} = \text{suffixorder}(\mathbf{x})$	Suffix OPP of $\mathbf{x}$
$\mathbf{b} = (b_1, \dots, b_i, \dots, b_{n-1})$	Binary sequence after time series $\mathbf{k}$ conversion
$\mathbf{d} = (d_1, \dots, d_j, \dots, d_{m-1})$	Binary sequence after pattern $\mathbf{s}$ conversion
$\mathbf{u} = (u_1, \dots, u_i, \dots, u_{m+1})$	$\mathbf{u} = \mathbf{o} \oplus \mathbf{f}$, superpattern with length $m+1$
$\mathbf{v} = (v_1, \dots, v_i, \dots, v_{m+1})$	$\mathbf{v} = \mathbf{o} \oplus \mathbf{f}$, superpattern with length $m+1$
$\mathbf{c} = (c_1, c_2, \dots, c_k)$	COP of pattern $\mathbf{o}$

4 PROPOSED ALGORITHM

Although OPP-Miner [52] and EFO-Miner [59] can discover all frequent patterns, for COP mining, it is inefficient to obtain all frequent patterns and then to discover COPs. To tackle COP mining, we propose COP-Miner, which has three parts: a local extremum point (LEP) algorithm to extract keypoints shown in Section 4.1, a preparation stage to realize the first round of mining shown in Section 4.2, and iteratively calculating supports and mining frequent COPs shown in Section 4.3. Fig. 2 shows the framework of COP-Miner and the COP-Miner algorithm and its time and space complexities analysis are shown in Section 4.4.

4.1 Extracting keypoints

Traditional OPP mining operates directly on the original time series to find sub-time series with the same trend. But there are two issues that should be considered. On the one hand, there is a lot of redundant information in the mining results. For example, OPP (1,2) indicates an upward trend, and OPP (1,2,3) also indicates a continuous upward trend. On the other hand, point-by-point analysis of time series cannot cope with sequences in noisy environments like scaling, warping, and shifting. Therefore, the first key task in COP mining is to extract keypoints. Hence, we extract keypoints according to Definition 2, and the pseudo-code of the local extremum point (LEP) algorithm is shown in Algorithm 1.

4.2 Preparation stage

This section is designed to prepare for the first round of mining. The related definitions and examples are as follows.

DEFINITION 10. (**Fusion pattern**) For a keypoint sub-time series $\mathbf{q}$ and its corresponding OPP $\mathbf{o} = R(\mathbf{q})$, if $\text{suffixorder}(\mathbf{o}) = \text{prefixorder}(\mathbf{f})$, then $\mathbf{f} = (f_1, f_2, \dots, f_{m-1}, f_m)$ is called a fusion pattern of OPP $\mathbf{o}$.

There are several methods to calculate the occurrences of the prefix OPP and suffix OPP.

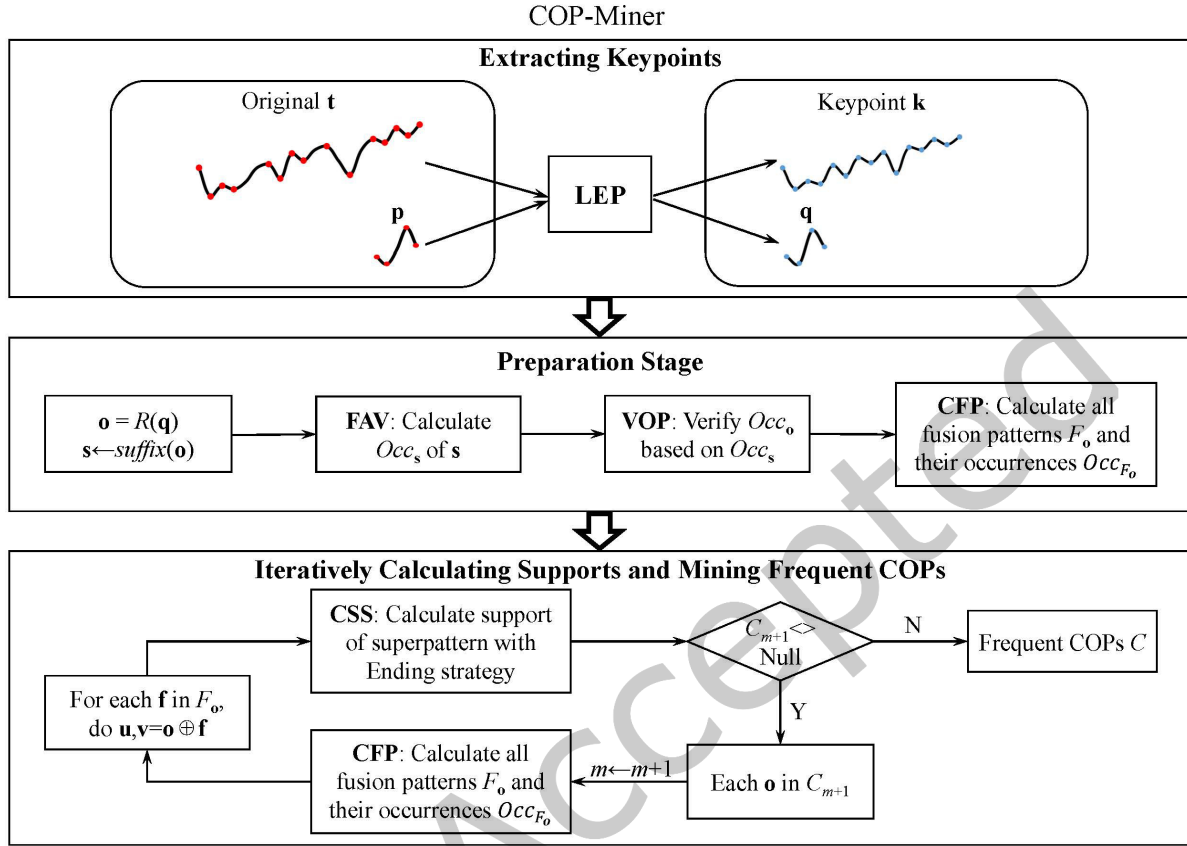


Fig. 2. Framework of COP-Miner. There are three parts: extracting keypoints, preparation stage, and iteratively calculating supports and mining frequent COPs whose key issue is to calculate supports of superpatterns.

Algorithm 1 LEP

Input: Time series t

Output: Keypoint time series k

- 1: Append t_1 to k ;
 - 2: **for** $i = 2$ to $N - 1$ **do**
 - 3: **if** $(t_i < t_{i-1} \wedge t_i \leq t_{i+1}) \vee (t_i \leq t_{i-1} \wedge t_i < t_{i+1})$ **then**
 - 4: Append t_i to k ;
 - 5: **else if** $(t_i > t_{i-1} \wedge t_i \geq t_{i+1}) \vee (t_i \geq t_{i-1} \wedge t_i > t_{i+1})$ **then**
 - 6: Append t_i to k ;
 - 7: **end if**
 - 8: **end for**
 - 9: Append t_N to k ;
 - 10: **return** k ;
-

- 1) A simple method is to enumerate all possible patterns and to use a pattern matching method to calculate the supports of these patterns. The principle of the enumeration method is as follows. We enumerate all fusion patterns. Suppose the length of $\text{suffixorder}(\mathbf{o})$ is $m-1$. It is easy to know there are m fusion patterns. Thus, we need to calculate the occurrences of $m+1$ patterns using pattern matching method. Obviously, this is a brute-force method.
- 2) An efficient method is to reduce the number of support calculations using the pattern matching method, since this is the most time-consuming step. To tackle this issue effectively, according to the principle of EFO-Miner [59], we need to calculate the occurrences of pattern $\text{suffixorder}(\mathbf{o})$ and its fusion patterns.

In this paper, we adopt the second method. For example, suppose the user specified a sub-time series (5,3,7,13,8). After extracting keypoints, the corresponding keypoint sub-time series is (5,3,13,8). $\mathbf{o} = R(\mathbf{q}) = (2,1,4,3)$, which indicates that $\mathbf{o}$ is the prefix pattern specified by user, and $\mathbf{o}$ is an OPP corresponding to keypoint sub-time series. Firstly, we need to obtain $\mathbf{o}$'s suffix OPP (1,3,2), since this is the common part of prefix pattern and its fusion patterns. Secondly, we employ the pattern matching method to calculate the occurrences of pattern (1,3,2). Finally, according to the principle of EFO-Miner [59], to calculate the supports of candidate superpatterns, we calculate the occurrences of the prefix pattern, i.e., the occurrences of pattern (2,1,4,3) and the occurrences of the suffix patterns, i.e., the occurrences of all fusion patterns of pattern (2,1,4,3).

Therefore, in the preparation stage, we have four steps: obtaining $\mathbf{s} = \text{suffixorder}(\mathbf{o})$, calculating the occurrences Occ_s of pattern $\mathbf{s}$, verifying the occurrences Occ_o of pattern $\mathbf{o}$ based on Occ_s , and calculating the occurrences of all fusion patterns of pattern $\mathbf{o}$.

Step 1. Obtain suffix OPP $\mathbf{s}$ of pattern $\mathbf{o}$ according to Definition 7.

EXAMPLE 6. Suppose we have a pattern $\mathbf{o} = (2,1,4,3)$. Thus, its suffix is (1,4,3), and the corresponding suffix OPP is $\mathbf{s} = (1,3,2)$.

Step 2. Use the filtration and verification (FAV) algorithm [52] to calculate the occurrences Occ_s of pattern $\mathbf{s}$. The FAV has three steps: filtration, SBNDM₂ matching, and verification. The filtration strategy is adopted to convert the time series and pattern into a binary sequence and a binary pattern. SBNDM₂ matching [66] is used to find the possible occurrences of a binary pattern in a binary sequence. The verification strategy is employed to check whether the possible occurrences are occurrences. This step is called filtration and verification (FAV).

Example 7 illustrates the principle of FAV.

EXAMPLE 7. Suppose we have the same keypoint time series $\mathbf{k}$ as in Example 2 and $\mathbf{s} = (1,3,2)$. Table 2 shows the element index of $\mathbf{k}$. According to the filtration strategy, keypoint time series $\mathbf{k}$ is transformed into a binary sequence $\mathbf{b}$. If $k_i < k_{i+1}$, then $b_i=1$; otherwise, $b_i=0$. For example, k_1 and k_2 are 16 and 8, respectively. Thus, $k_1 > k_2$, i.e., $b_1=0$. Similarly, $b_2=1$. Finally, $\mathbf{b} = (0,1,0,1,0,1,0,1,0,1,0,1,0,1,0,1)$. Moreover, $s_1 < s_2$, since s_1 and s_2 are 1 and 3, respectively. Thus, $d_1=1$. Similarly, $d_2=0$. Therefore, $\mathbf{s}$ is transformed into $\mathbf{d} = (1,0)$. Then, according to SBNDM₂ matching, we know that $\langle 3 \rangle$ is a possible occurrence of pattern $\mathbf{d}$ in sequence $\mathbf{b}$. Similarly, $\langle 5 \rangle$, $\langle 7 \rangle$, $\langle 9 \rangle$, $\langle 11 \rangle$, and $\langle 13 \rangle$ are also possible occurrences. Finally, according to the verification strategy, we verify whether $\langle 3 \rangle$ is an occurrence. For pattern $\mathbf{d} = (1,0)$, $\langle 3 \rangle$ in $\mathbf{b}$ means (b_2, b_3) whose corresponding sub-time series in $\mathbf{k}$ is (k_2, k_3, k_4) . $\langle 4 \rangle$ is the occurrence of pattern $\mathbf{s}$ in $\mathbf{k}$, since $k_2 < k_4$, which is consistent with $s_1 < s_3$. Similarly, $\langle 6 \rangle$, $\langle 8 \rangle$, $\langle 12 \rangle$, and $\langle 14 \rangle$ are also occurrences of pattern $\mathbf{s}$, while $\langle 10 \rangle$ is not.

Table 2. Element index of $\mathbf{k}$

Index	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$\mathbf{k}$	16	8	11	10	17	13	20	18	22	14	24	23	27	25	28

Step 3. Verify the occurrences $Occ_{\mathbf{o}}$ of pattern $\mathbf{o}$ based on $Occ_{\mathbf{s}}$. We know that $\mathbf{s} = (s_1, s_2, \dots, s_{m-1})$ is the suffix pattern of $\mathbf{o} = (o_1, o_2, \dots, o_m)$. Suppose $\langle c \rangle$ is an occurrence of $\mathbf{s}$, which means that the OPP of sub-time series $(k_{c-m+2}, k_{c-m+3}, \dots, k_c)$ is $R(\mathbf{s})$. Now, we verify whether $\langle c \rangle$ is also an occurrence of $\mathbf{o}$, which means verifying whether the OPP of sub-time series $(k_{c-m+1}, k_{c-m+2}, k_{c-m+3}, \dots, k_c)$ is $\mathbf{o}$ based on $R(\mathbf{s})$. To verify it easily, we propose the lower position and upper position at first.

DEFINITION 11. (Lower position and upper position) We compare the values in $(s_1, s_2, \dots, s_{m-1})$ and $(q_2, \dots, q_{m-1}, q_m)$ correspondingly. If $s_j = q_{j+1}$, then $j+1$ is an unchanged position; otherwise, $j+1$ is a changed position. Among the unchanged positions, if the value in l_p is the maximum value, then l_p is called the lower position. Among the changed positions, if the value in u_p is the minimum value, then u_p is called the upper position.

EXAMPLE 8. In Example 6, we know $\mathbf{s} = (1, 3, 2)$ and $\mathbf{o} = (2, 1, 4, 3)$. Now, we compare the values in $(1, 3, 2)$ and $(1, 4, 3)$ correspondingly. Obviously, 2 is an unchanged position, since $s_1 = o_2 = 1$. Moreover, 2 is the lower position, since 2 is the only unchanged position. Similarly, $\{3, 4\}$ are changed positions, since $s_2 = 3 \neq o_3 = 4$ and $s_3 = 2 \neq o_4 = 3$. Among the changed positions $\{3, 4\}$, the value in 4 is the minimum value, since $o_4 = 3 < o_3 = 4$. Thus, 4 is the upper position.

Then, based on lower and upper positions, we verify whether $k_{c-m+1} > k_{c-m+l_p}$ and $k_{c-m+1} < k_{c-m+u_p}$ are true. If it is true, then $\langle c \rangle$ is also an occurrence of $\mathbf{o}$; otherwise, $\langle c \rangle$ is not. It is worth noting that k_{c-m+l_p} or k_{c-m+u_p} can be null. If it is null, the corresponding comparison is pruned.

EXAMPLE 9. In Example 8, we know that 2 and 4 are the lower and upper positions, respectively. Moreover, the length of pattern $\mathbf{s} = (1, 3, 2)$ is 3. Thus, $m-3+1=4$. For occurrence $\langle 4 \rangle$ in Example 7, $c-m+1=1$. Then, we verify whether $k_1 > k_2$ and $k_1 < k_4$ are true or not. $\langle 4 \rangle$ is not an occurrence of pattern $\mathbf{o}$ in $\mathbf{k}$, since $k_1 > k_4$. Similarly, for occurrence $\langle 6 \rangle$, we verify whether $k_3 > k_4$ and $k_3 < k_6$. We know that $\langle 6 \rangle$ is an occurrence, since $k_3 = 11$, $k_4 = 10$, and $k_6 = 13$. Moreover, $\langle 8 \rangle$, $\langle 12 \rangle$, and $\langle 14 \rangle$ are three occurrences of $\mathbf{o}$ in $\mathbf{k}$. Now, we validate that k_{c-m+l_p} or k_{c-m+u_p} can be null. Suppose $\mathbf{o} = (4, 1, 3, 2)$ and $\mathbf{s} = (1, 3, 2)$. Thus, $\{2, 3, 4\}$ are unchanged positions, while the changed position is null. Therefore, 3 is the lower position and the upper position is null, which means that we only need to verify whether $k_{c-m+1} > k_{c-m+l_p}$.

The pseudo-code of verification occurrences for pattern (VOP) $\mathbf{o}$ is shown in Algorithm 2.

Algorithm 2 VOP

Input: OPP $\mathbf{o}$ and its suffix pattern $\mathbf{s}$, occurrences of $\mathbf{s}$: $Occ_{\mathbf{s}}$, keypoint time series $\mathbf{k}$

Output: Occurrences of $\mathbf{o}$: $Occ_{\mathbf{o}}$

```

1: Calculate  $l_p$  and  $u_p$  according to Definition 10;
2:  $m \leftarrow |\mathbf{s}| + 1$ ; //  $|\mathbf{s}|$  is the length of  $\mathbf{s}$ 
3: for each  $c$  in  $Occ_{\mathbf{s}}$  do
4:   if  $k_{c-m+1} > k_{c-m+l_p} \wedge k_{c-m+1} < k_{c-m+u_p}$  then
5:     Append  $c$  to  $Occ_{\mathbf{o}}$ ;
6:   end if
7: end for
8: return  $Occ_{\mathbf{o}}$ ;

```

Step 4. Calculate the occurrences of all fusion patterns of pattern $\mathbf{o}$. In this step, the main issue is that given pattern $\mathbf{s}$, sub-time series $(k_{c-m+2}, k_{c-m+3}, \dots, k_c)$ according to occurrence $\langle c \rangle$, and new value k_{c+1} , we calculate the OPP of $\mathbf{k}' = (k_{c-m+2}, k_{c-m+3}, \dots, k_c, k_{c+1})$, i.e., $R(\mathbf{k}')$. An easy method is to adopt a linear search method by comparing k_{c+1} with k_{c-i} ($0 \leq i \leq m-2$) to obtain $\mathbf{f} = (f_1, f_2, \dots, f_{m-1}, f_m)$ based on $(s_1, s_2, \dots, s_{m-1})$. Obviously, this method is inefficient. An efficient method is to employ the binary search method to calculate the relative order of

k_{c+1} , i.e., $v = o_{\mathbf{k}'}(k_{c+1})$, which means that k_{c+1} is the v -th smallest in $\mathbf{k}'$. Furthermore, each $\langle c \rangle$ in $Occ_{\mathbf{s}}$ is calculated according to the binary search method. All fusion patterns $F_{\mathbf{o},v}$ are stored into a set $F_{\mathbf{o}}$, and corresponding occurrences $Occ_{F_{\mathbf{o},v}}$ are stored into $Occ_{F_{\mathbf{o}}}$.

EXAMPLE 10. We use Table 2 and Example 7 to illustrate this example. We know that $\mathbf{s} = (1,3,2)$ and $Occ_{\mathbf{s}} = \{4,6,8,12,14\}$. Taking $\langle 4 \rangle$ as an example, we calculate the OPP of $(8,11,10,17)$. We can employ the binary search method, since $(1,3,2)$ is the relative order of $(8,11,10)$. It is easy to know that the rank order of 17 in $(8,11,10,17)$ is 4. Thus, $\langle 5 \rangle$ as an occurrence of $(1,3,2,4)$ is stored into $Occ_{F_{\mathbf{o},4}}$. Similarly, we know that the rank orders of k_7, k_9, k_{13} , and k_{15} are 4. Hence, $Occ_{F_{\mathbf{o},4}} = \{5,7,9,13,15\}$, and $F_{\mathbf{o},4}$ is $(1,3,2,4)$.

The pseudo-code of calculating fusion patterns (CFP) is shown in Algorithm 3.

Algorithm 3 CFP

Input: Suffix pattern $\mathbf{s}$ of $\mathbf{o}$, occurrences of $\mathbf{s}$: $Occ_{\mathbf{s}}$, keypoint time series $\mathbf{k}$

Output: Fusion patterns $F_{\mathbf{o}}$, occurrences of $F_{\mathbf{o}}$: $Occ_{F_{\mathbf{o}}}$

```

1: for each  $c$  in  $Occ_{\mathbf{s}}$  do
2:    $v \leftarrow \text{Binary\_search}(\mathbf{s}, c, k_{c+1})$ ;
3:    $F_{\mathbf{o},v}++$ ;
4:   Append  $\langle c+1 \rangle$  to  $Occ_{F_{\mathbf{o},v}}$ ;
5: end for
6: return  $F_{\mathbf{o}}$  and  $Occ_{F_{\mathbf{o}}}$ ;

```

4.3 Calculating supports of superpatterns

Now, we know the occurrences of prefix OPP and suffix OPP. To calculate the supports of superpatterns, we propose the algorithm CSS based on pattern fusion whose principle is shown as follows.

Suppose $Occ_{\mathbf{o}} = \{go_1, \dots, go_i, \dots, go_y\}$ and $Occ_{\mathbf{f}} = \{gf_1, \dots, gf_j, \dots, gf_z\}$ are occurrences of OPPs $\mathbf{o} = (o_1, o_2, \dots, o_m)$ and $\mathbf{f} = (f_1, f_2, \dots, f_m)$, respectively. There are two cases for pattern $\mathbf{o}$ fused with pattern $\mathbf{f}$.

Case 1: If $o_1 = f_m$, then two superpatterns with length $m+1$, $\mathbf{u} = (u_1, \dots, u_i, \dots, u_{m+1})$ and $\mathbf{v} = (v_1, \dots, v_i, \dots, v_{m+1})$ are generated by $\mathbf{u}, \mathbf{v} = \mathbf{o} \oplus \mathbf{f}$, where $u_1 = v_{m+1} = o_1$ and $u_{m+1} = v_1 = o_1 + 1$. For $1 \leq i \leq m-1$, if $f_i < o_1$, then $u_{i+1} = v_{i+1} = f_i$; otherwise, if $f_i > o_1$, then $u_{i+1} = v_{i+1} = f_i + 1$. Furthermore, the occurrences of $\mathbf{u}$ and $\mathbf{v}$ are calculated as follows: $\langle gf_j \rangle$ may be an occurrence of $\mathbf{u}$ or $\mathbf{v}$, iff $gf_j = go_i + 1$. Then, we need to determine the values of k_{begin} and k_{end} , where $begin = gf_j - m$ and $end = gf_j$.

(1) If $k_{begin} = k_{end}$, then $\langle gf_j \rangle$ is not an occurrence of any superpattern.

(2) If $k_{begin} < k_{end}$, then $\langle gf_j \rangle$ is an occurrence of $\mathbf{u}$, i.e., $\langle gf_j \rangle$ is added into $Occ_{\mathbf{u}}$, and $\langle go_i \rangle$ and $\langle gf_j \rangle$ are pruned from $Occ_{\mathbf{o}}$ and $Occ_{\mathbf{f}}$, respectively.

(3) If $k_{begin} > k_{end}$, then $\langle gf_j \rangle$ is an occurrence of $\mathbf{v}$, i.e., $\langle gf_j \rangle$ is added into $Occ_{\mathbf{v}}$, and $\langle go_i \rangle$ and $\langle of_j \rangle$ are pruned from $Occ_{\mathbf{o}}$ and $Occ_{\mathbf{f}}$, respectively.

Case 2: If $o_1 \neq f_m$, then a superpattern with length $m+1$, $\mathbf{u} = (u_1, \dots, u_i, \dots, u_{m+1})$ is generated by $\mathbf{u} = \mathbf{o} \oplus \mathbf{f}$.

(1) If $o_1 < f_m$, then $u_1 = o_1$. For $1 \leq i \leq m$, if $f_i < o_1$, then $u_{i+1} = f_i$; otherwise, $u_{i+1} = f_i + 1$.

(2) If $o_1 > f_m$, then $u_1 = o_1 + 1$. For $1 \leq i \leq m$, if $f_i \leq o_1$, then $u_{i+1} = f_i$; otherwise, $u_{i+1} = f_i + 1$.

Moreover, the occurrences of pattern $\mathbf{u}$ are calculated as follows: $\langle gf_j \rangle$ is an occurrence of $\mathbf{u}$, iff $gf_j = go_i + 1$, i.e., $\langle gf_j \rangle$ is added into $Occ_{\mathbf{u}}$, and $\langle go_i \rangle$ and $\langle gf_j \rangle$ are pruned from $Occ_{\mathbf{o}}$ and $Occ_{\mathbf{f}}$, respectively. $sup(\mathbf{u}, \mathbf{k}) = |Occ_{\mathbf{u}}|$.

According to Cases 1 and 2, we know that if we find an occurrence of the superpattern, then an element will be pruned from $Occ_{\mathbf{o}}$. Therefore, the size of $Occ_{\mathbf{o}}$ gradually decreases. To further improve efficiency, we propose an ending strategy.

Ending strategy. If $|Occ_o| < minsup$, then CSS is terminated.

Example 11 illustrates the principle of CSS.

EXAMPLE 11. In this example, we use the results of Examples 9 and 10. We know that $o = (2,1,4,3)$ and $Occ_o = \{6,8,12,14\}$, $F_o = \{(1,3,2,4)\}$ and $Occ_{(1,3,2,4)} = \{5,7,9,13,15\}$. Pattern o can be fused with each frequent fusion pattern in F_o using CSS. If $minsup=3$, then a pattern that does not meet the threshold can be pruned. Suppose $f = F_{o,4} = (1,3,2,4)$. Patterns o and f can be fused into one pattern, i.e., $o \oplus f = u = (2,1,4,3,5)$, since $o_1 \neq f_4$, which belongs to Case 2. We know the first element of Occ_o is 6, and 7 is in Occ_f . Thus, $\langle 7 \rangle$ is an occurrence of u , i.e., $\langle 7 \rangle$ is added into Occ_u , and $\langle 6 \rangle$ and $\langle 7 \rangle$ are pruned from Occ_o and Occ_f , respectively. Similarly, $\langle 9 \rangle$, $\langle 13 \rangle$, and $\langle 15 \rangle$ are also three occurrences of u ; $\langle 9 \rangle$, $\langle 13 \rangle$, and $\langle 15 \rangle$ are added into Occ_u , i.e., $Occ_u = \{7,9,13,15\}$ and $sup(u, k)=4$; $\langle 8 \rangle$, $\langle 12 \rangle$, and $\langle 14 \rangle$ are pruned from Occ_o ; $\langle 9 \rangle$, $\langle 13 \rangle$, and $\langle 15 \rangle$ are pruned from Occ_f , i.e., $Occ_o = \{\emptyset\}$ and $Occ_f = \{5\}$. Hence, $u = (2,1,4,3,5)$ is a frequent COP with length 5, since $sup(u, k)=4$. Now, CSS is terminated, since the size of Occ_o is zero.

Now, we prove that the ending strategy is correct. Before this, we first show that COP mining satisfies anti-monotonicity.

THEOREM 1. COP mining satisfies anti-monotonicity, that is, the support of a superpattern is not greater than that of its subpattern.

PROOF. Suppose $\langle gx_j \rangle$ is an occurrence of superpattern $x = (x_1, x_2, \dots, x_m)$. Then we can safely say that $\langle gx_j - 1 \rangle$ is an occurrence of subpattern $(x_1, x_2, \dots, x_{m-1})$ according to Cases 1 and 2. Therefore, the number of occurrences of a superpattern is not greater than that of its subpattern, i.e., the support of a superpattern is not greater than that of its subpattern. Hence, COP mining satisfies anti-monotonicity. $\square$

THEOREM 2. The ending strategy is correct.

PROOF. When pattern o fuses with pattern f , if an occurrence of superpattern u or v is found, then the corresponding occurrence of Occ_o is removed, which means that the size of Occ_o gradually decreases. We know that COP mining satisfies anti-monotonicity. Therefore, when $|Occ_o| < minsup$, if pattern o fuses with a new pattern w , then the support of superpattern is also less than $minsup$. Hence, CSS can be terminated, which means that we do not need to fuse pattern o with the other patterns in F_o . $\square$

The pseudo-code of CSS is shown in Algorithm 4.

4.4 COP-Miner

To mine all COPs of the given pattern $o = R(q)$, we propose COP-Miner, whose main steps are as follows.

Step 1: Extract keypoints of time series t and sub-time series p using the LEP algorithm, and then obtain keypoint time series k and keypoint sub-time series q .

Step 2: Adopt the method of preparation stage to calculate the occurrences of pattern o and further generate all fusion patterns F_o , thereby calculating the occurrences of all fusion patterns of pattern o .

Step 3: Use the CSS algorithm to calculate the supports of superpatterns, and then mine frequent COPs C_{m+1} with length $m+1$.

Step 4: For each frequent pattern h in C_k , adopt the CFP algorithm to calculate the occurrences of all fusion patterns of pattern h .

Step 5: Iterate Steps 3 and 4, until C_{k+1} is empty.

Example 12 is used to illustrate the principle of COP-Miner.

EXAMPLE 12. Suppose we have the same time series t as in Example 2 and pattern $p = (5,3,7,13,8)$, and we set $minsup=3$.

Algorithm 4 CSS

Input: Keypoint time series $\mathbf{k}$, OPP $\mathbf{o}$ of keypoint sub-time series $\mathbf{q}$, occurrences of $\mathbf{o}$: $Occ_{\mathbf{o}}$, fusion patterns $F_{\mathbf{o}}$, occurrences of $F_{\mathbf{o}}$: $Occ_{F_{\mathbf{o}}}$, and $minsup$

Output: Frequent COP set C_{m+1} and their occurrences $Occ_{C_{m+1}}$

```

1: for each pattern  $\mathbf{f}$  in  $F_{\mathbf{o}}$  do
2:   if  $|Occ_{\mathbf{f}}| \geq minsup$  then
3:     if  $\mathbf{o}[1] == \mathbf{f}[m]$  then
4:        $\mathbf{u}, \mathbf{v} = \mathbf{o} \oplus \mathbf{f}$ ;
5:       Obtain occurrences  $Occ_{\mathbf{u}}$  and  $Occ_{\mathbf{v}}$  according to Case 1, and update  $Occ_{\mathbf{o}}$  and  $Occ_{\mathbf{f}}$ ;
6:       If  $\mathbf{u}$  and  $\mathbf{v}$  are frequent, then store them into  $C_{m+1}$ ;
7:     else
8:        $\mathbf{u} = \mathbf{o} \oplus \mathbf{f}$ ;
9:       Obtain occurrences  $Occ_{\mathbf{u}}$  according to Case 2, and update  $Occ_{\mathbf{o}}$  and  $Occ_{\mathbf{f}}$ ;
10:      If  $\mathbf{u}$  is frequent, then store it into  $C_{m+1}$ ;
11:    end if
12:  end if
13:  if  $|Occ_{\mathbf{o}}| < minsup$  then
14:    break; // Ending strategy
15:  end if
16: end for
17: return  $C_{m+1}$  and  $Occ_{C_{m+1}}$ ;

```

According to Step 1, we obtain keypoint time series $\mathbf{k}$ shown in Table 2 and keypoint sub-time series $\mathbf{q} = (5, 3, 13, 8)$ using the LEP algorithm.

According to Step 2, the OPP of $\mathbf{q}$ is $\mathbf{o} = R(\mathbf{q}) = (2, 1, 4, 3)$, and we adopt the method of preparation stage to calculate the occurrences $Occ_{\mathbf{o}} = \{6, 8, 12, 14\}$. There is one fusion pattern $F_{\mathbf{o},4} = (1, 3, 2, 4)$, whose occurrences is $Occ_{F_{\mathbf{o},4}} = \{5, 7, 9, 13, 15\}$.

According to Step 3, pattern $\mathbf{o}$ and fusion pattern $F_{\mathbf{o},4}$ can be fused into a superpattern $\mathbf{u} = (2, 1, 4, 3, 5)$ and $Occ_{\mathbf{u}} = \{7, 9, 13, 15\}$ using the CSS algorithm.

According to Step 4, $\mathbf{u}$ is the only frequent COP, and we store $\mathbf{u}$ into C_5 . Using the CFP algorithm, we discover two fusion patterns $F_{\mathbf{u},2} = (1, 4, 3, 5, 2)$ and $F_{\mathbf{u},4} = (1, 3, 2, 5, 4)$, whose occurrences are $Occ_{F_{\mathbf{u},2}} = \{10\}$ and $Occ_{F_{\mathbf{u},4}} = \{8, 14\}$. C_6 is empty, since these two fusion patterns of pattern $\mathbf{u}$ are infrequent.

Therefore, all frequent COPs of pattern $\mathbf{q}$ are in $C = \{(2, 1, 4, 3, 5)\}$.

The pseudo-code of COP-Miner is shown in Algorithm 5.

THEOREM 3. The time complexity of COP-Miner is $O(N + M + n \times \log_2^m + d \times (n + m))$, where N , M , n , m , and d are the lengths of the original time series t , the given prefix pattern p , the keypoint time series $\mathbf{k}$, the keypoint pattern $\mathbf{q}$, and the number of iterations of Line 8 in COP-Miner which means that the maximum length of the mined pattern is $d+m$, respectively.

PROOF. Line 1 of COP-Miner extracts the keypoints of the original time series and the given prefix pattern. The time complexity of Line 1 is $O(N + M)$. Line 2 calculates $\text{suffixorder}(\mathbf{q})$ whose time complexity is $O(m)$. The time complexity of generating binary sequences in FAV is $O(n + m)$. The time complexity of SBNDM₂ matching in FAV is $O(n \times \log_2^m)$. The time complexity of verification in FAV is $O(n)$. Thus, the time complexity of Line 3 is $O(n \times \log_2^m + m)$. The maximal length of Occs is less than n . Thus, the time complexity of Line 4 (the VOP algorithm) is $O(n)$. Moreover, the time complexity of Line 2 in the CFP algorithm is $O(\log_2^m)$. Therefore, the time complexity of Line 5 (the CFP algorithm) is $O(n \times \log_2^m)$. The CSS algorithm has two parts: generating

Algorithm 5 COP-Miner

Input: Time series $\mathbf{t}$, pattern $\mathbf{p}$, $minsup$
Output: Frequent COPs C

- 1: $\mathbf{k} \leftarrow \text{LEP}(\mathbf{t})$, $\mathbf{q} \leftarrow \text{LEP}(\mathbf{p})$; //Preparation stage
- 2: $\mathbf{o} = R(\mathbf{q})$, calculate $\mathbf{s} = \text{suffixorder}(\mathbf{o})$ according to Definition 7;
- 3: Use FAV to calculate the occurrences $Occ_{\mathbf{s}}$;
- 4: $Occ_{\mathbf{o}} \leftarrow \text{VOP}(\mathbf{o}, \mathbf{s}, Occ_{\mathbf{s}}, \mathbf{k})$;
- 5: $F_{\mathbf{o}}, Occ_{F_{\mathbf{o}}} \leftarrow \text{CFP}(\mathbf{o}, Occ_{\mathbf{s}}, \mathbf{k})$;
 //Iteratively calculating supports and mining frequent COPs
- 6: $C_{m+1}, Occ_{C_{m+1}} \leftarrow \text{CSS}(\mathbf{k}, \mathbf{o}, Occ_{\mathbf{o}}, F_{\mathbf{o}}, Occ_{F_{\mathbf{o}}}, minsup)$;
- 7: $g \leftarrow m+1$, $C \leftarrow C_{m+1}$;
- 8: **while** $C_g \neq \text{null}$ **do**
- 9: **for each** $\mathbf{h}$ in C_g **do**
- 10: $F_{\mathbf{h}}, Occ_{F_{\mathbf{h}}} \leftarrow \text{CFP}(\mathbf{h}, Occ_{\text{suffixorder}(\mathbf{h})}, \mathbf{k})$;
- 11: $C_{g+1}, Occ_{C_{g+1}} \leftarrow \text{CSS}(\mathbf{k}, \mathbf{h}, Occ_{\mathbf{h}}, F_{\mathbf{h}}, Occ_{F_{\mathbf{h}}}, minsup)$;
- 12: $C \leftarrow C \cup C_{g+1}$;
- 13: **end for**
- 14: $g \leftarrow g+1$;
- 15: **end while**
- 16: **return** C ;

superpatterns and calculating their occurrences. Thus, the time complexity of Line 6 (the CSS algorithm) is $O(n + m)$. The time complexity of Lines 8 to 15 is $O(d \times (n + m))$, since the number of iterations of Line 8 is d . Hence, the time complexity of COP-Miner is $O(N + M + n \times \log_2^m + d \times (n + m))$. $\square$

THEOREM 4. *The space complexity of COP-Miner is $O(N + M + d \times (n + m))$.*

PROOF. The space complexity of Line 1 of COP-Miner is $O(N + M + n + m)$, since LEP converts $\mathbf{t}$ and $\mathbf{p}$ into $\mathbf{k}$ and $\mathbf{q}$, respectively. The space complexity of Line 2 is $O(m)$. The space complexity of Line 3 is $O(n)$. The space complexity of Line 4 (the VOP algorithm) is $O(n)$. The space complexity of Line 5 (the CFP algorithm) is $O(n + m)$. Moreover, the space complexity of Lines 8 to 15 is $O(d \times (n + m))$. Hence, the space complexity of COP-Miner is $O(N + M + d \times (n + m))$. $\square$

5 EXPERIMENTAL RESULTS AND ANALYSIS

In this section, we consider the performance of COP-Miner from multiple perspectives. Moreover, we answer the following five research questions (RQs) through experiments:

- RQ1: Do we need to specially design an algorithm?
- RQ2: What is the effect of each strategy in COP-Miner?
- RQ3: Does COP-Miner have better scalability than other competing algorithms?
- RQ4: How do different parameters affect the performance of COP-Miner?
- RQ5: What is the impact of keypoint extraction on time series prediction?

To answer RQ1, we compared the efficiency of COP-Miner with OPP-Miner and EFO-Miner, which mine all OPPs and then filter COPs. To address RQ2, we proposed COP-kom, COP-noVOP, COP-noCFP, COP-efo, COP-enum, COP-noending, and FAV-sliding to verify the efficiency of COP-Miner (see Section 5.3). To answer

RQ3, we conducted experiments on datasets of different lengths to report the performance of scalability (see Section 5.4). To address RQ4, we explored the effects of prefix patterns with different lengths and minimum support thresholds on the running time of the algorithm (see Section 5.5). To solve RQ5, we selected two models: autoregressive integrated moving average (ARIMA) [4, 27], dynamic time warping (DTW) [19], and proposed COP-noLEP to verify the prediction performance (see Section 5.6).

5.1 Datasets

We used nine real datasets for experimental evaluation. The electrocardiogram (ECG) datasets include KURIAS_HeartRate, KURIAS_PAxis, and KURIAS_PRIInterval. They are part of a 12-lead ECG database with standardized diagnostic ontology, which can be downloaded at <https://physionet.org/content/kurias-ecg/1.0/KURIAS-ECG>. The stock datasets are the opening price of the S&P 500 and listed stocks on the New York Stock Exchange, which can be downloaded at <https://www.yahoo.com/>. To validate the performance of COP-Miner on large-scale dataset, we generated a synthetic real dataset named Big_NYSE by enlarging SDB8 440 times. The PM2.5 and temperature datasets are the records of Chengdu and Guangzhou, respectively, which can be downloaded at <https://archive.ics.uci.edu/ml/datasets.php>. The CSSE COVID-19 dataset is the information on the number of confirmed cases in Alabama, USA, which can be downloaded at <https://datahub.io/core/covid-19>. The specific description of each dataset is shown in Table 3.

Table 3. Description of datasets

Dataset	Name	Type	Total length
SDB1	KURIAS_HeartRate	ECG	13,863
SDB2	KURIAS_PAxis	ECG	13,863
SDB3	KURIAS_PRIInterval	ECG	13,863
SDB4	S&P 500	Stock	23,046
SDB5	ChengduPM2.5	PM2.5	28,900
SDB6	GuangZhouTemp	Temperature	52,582
SDB7	US_Alabama_Confirmed	CSSE COVID-19	56,304
SDB8	NYSE	Stock	60,000
SDB9	Big_NYSE	Stock	26,400,000

We ran the experiments presented in this paper on a computer with Intel(R) Core(TM) i5-5200U, 2.20GHZ CPU, 4.0GB RAM, and the Windows 10 64-bit operating system. The programming environment was IntelliJ IDEA 2019.2. All the algorithms and datasets can be downloaded from <https://github.com/wuc567/Pattern-Mining/tree/master/COP-Miner>.

5.2 Baseline methods

To evaluate the efficiency and prediction performance of the COP-Miner algorithm, we designed four experiments and selected ten competing algorithms for comparison.

1. OPP-Miner [52] and EFO-Miner [59]: To validate that it is necessary to design a special algorithm, we applied OPP-Miner and EFO-Miner, which adopt a pattern matching strategy and subpattern's matching results to calculate the support of superpattern, respectively. They both generate COPs based on find-all-frequent OPPs.

2. COP-kom: To validate the efficiency of the FAV [52] algorithm, we proposed COP-kom, which applies the KMP-Order-Matcher [20] algorithm to obtain the occurrences of *suffixorder*(**q**).

3. COP-noVOP: To validate the efficiency of the VOP algorithm, we developed COP-noVOP, which does not use the VOP algorithm, but rather applies a linear search to obtain the occurrences of pattern **q**.

4. COP-noCFP: To validate the efficiency of CFP algorithm, we developed COP-noCFP, which does not use a binary search, but rather applies a linear search to calculate the occurrences of all fusion patterns.
5. COP-efo: To evaluate the superiority of the method of the preparation stage, we proposed COP-efo, which employs EFO-Miner [59] to calculate the occurrences of prefix pattern q and its fusion patterns.
6. COP-enum: To analyze the performance of pattern fusion strategy in CSS, we proposed COP-enum algorithm, which generates candidate patterns using an enumeration method.
7. COP-noending: To verify the effectiveness of the ending strategy, we developed COP-noending, which adopts the CSS algorithm to calculate the support, but does not use the ending strategy.
8. FAV-sliding: To verify the efficiency of the CSS algorithm, we designed FAV-sliding, which employs FAV [52] to calculate the occurrences Occ_s of pattern s , and then uses the sliding windows to calculate the occurrences of superpatterns.
9. COP-noLEP: To verify the influence of extracting keypoints on the prediction performance, we developed COP-noLEP to generate COPs, which does not use the LEP algorithm, but rather mines COPs directly from the original time series.

5.3 Performance of COP-Miner

To determine the necessity of specially designing COP-Miner, we selected OPP-Miner and EFO-Miner as the competing algorithms. To evaluate the performance of the FAV, VOP, and CFP algorithms, we employed COP-kom, COP-noVOP, and COP-noCFP as competing algorithms. To evaluate the performance of the method of the preparation stage, we designed COP-efo as a competing algorithm. To evaluate the performance of the pattern fusion strategy and ending strategy, we employed COP-enum and COP-noending as competing algorithms. Moreover, to examine the performance of the CSS algorithm, we designed FAV-sliding as a competing algorithm. We set $minsup=10$ on SDB1-SDB8 and set $minsup=6000$ on SDB9. Since all ten algorithms are complete, they mine the same number of frequent patterns. When the length of a given pattern is six, i.e., $P_{SDB1} = P_{SDB3} = (1,5,2,6,3,4)$, $P_{SDB2} = (1,5,3,4,2,6)$, $P_{SDB4} = (1,4,3,6,5,2)$, $P_{SDB5} = P_{SDB7} = P_{SDB8} = P_{SDB9} = (1,3,2,5,4,6)$, and $P_{SDB6} = (3,6,4,5,1,2)$, there are 5, 5, 6, 24, 21, 2, 2, 82, and 52 frequent COPs mined from SDB1-SDB9, respectively. Figs. 3 and 4 show the comparisons of running time and memory usage, respectively. We also show the comparisons of the numbers of candidate patterns and the number of occurrences of superpatterns in Figs. 5 and 6, respectively. It is worth noting that the information regarding the OPP-Miner and FAV-sliding algorithms are not presented in Fig. 6, since they do not use the occurrences of subpatterns to calculate the support of superpattern.

The results give rise to the following observations.

1. COP-Miner runs faster and costs less memory than both OPP-Miner and EFO-Miner. For example, on SDB9, it can be seen from Fig. 3 that OPP-Miner and EFO-Miner require 11,179.622s and 1,053.261s of running time, respectively, while COP-Miner needs 280.130s. Thus, COP-Miner is about 40 times and 4 times faster than OPP-Miner and EFO-Miner, respectively. In Fig. 4, OPP-Miner and EFO-Miner occupy 8,191.4Mb and 6,509.3Mb of running memory, respectively, while COP-Miner occupies only 2,099.8Mb. The reason is as follows. Both OPP-Miner and EFO-Miner discover COPs based on finding all frequent OPPs, which is a brute-force method to discover COPs. Thus, the two algorithms have to check more candidate patterns and generate more occurrences of superpatterns. Figs. 5 and 6 also verify these claims. From Fig. 5, OPP-Miner and EFO-Miner generate 12,153 and 10,404 candidate patterns, respectively, while COP-Miner generates only 64. From Fig. 6, EFO-Miner makes 441,900,345 comparisons while COP-Miner makes only 1,553,928. We can find the same effect on other datasets. Therefore, COP-Miner outperforms OPP-Miner and EFO-Miner, and it is necessary to specially design an algorithm for COP mining.
2. COP-Miner runs faster and requires less memory than COP-kom, COP-noVOP, and COP-noCFP. For example, on SDB4, it can be seen from Fig. 3 that COP-kom, COP-noVOP, and COP-noCFP require 0.400s, 0.380s, and 0.392s,

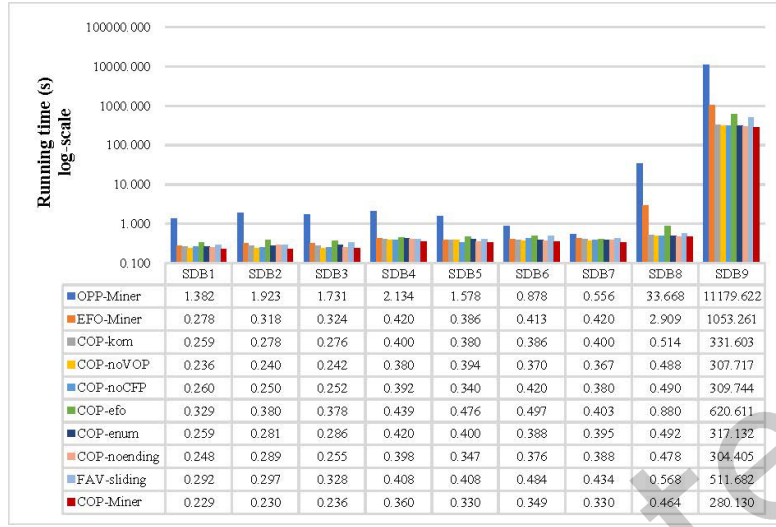


Fig. 3. Comparison of running time on SDB1-SDB9

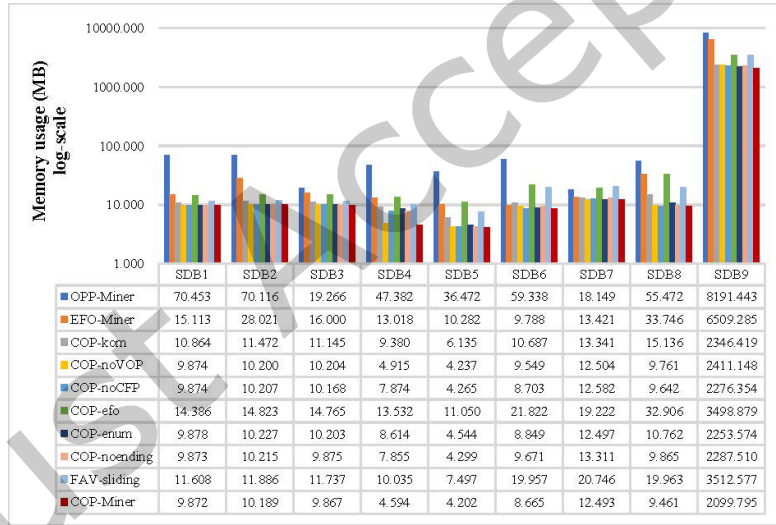


Fig. 4. Comparison of memory usage on SDB1-SDB9

respectively, while COP-Miner only needs 0.360s. From Fig. 4, COP-kom, COP-noVOP, and COP-noCFP occupy 9.380Mb, 4.915Mb, and 7.874Mb, respectively, while COP-Miner occupies only 4.594Mb. The reasons are as follows. The four algorithms employ the same CSS algorithm to calculate the supports of superpatterns. Thus, from Figs. 5 and 6, we know that the four algorithms generate the same number of candidate patterns and occurrences of superpatterns. For example, on SDB4, the four algorithms generate 57 candidate patterns and make 3,860 comparisons. The difference between COP-kom and COP-Miner is that COP-kom uses the KMP-Order-Matcher [20] algorithm to find the occurrences of $\text{suffixorder}(\mathbf{q})$, while COP-Miner adopts the FAV algorithm. The results

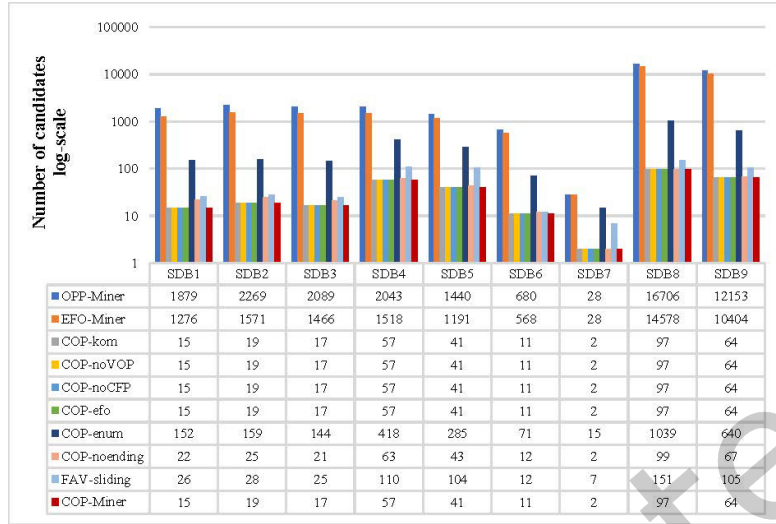


Fig. 5. Comparison of numbers of candidate patterns on SDB1-SDB9



Fig. 6. Comparison of numbers of occurrences of superpatterns on SDB1-SDB9

indicate that FAV is more effective than KMP-Order-Matcher. Similarly, the results validate that the designed VOP algorithm is more efficient than the linear search to obtain the occurrences of pattern q , and the binary search in CFP is more effective than the linear search to calculate the occurrences of fusion patterns. Therefore, COP-Miner outperforms COP-kom, COP-noVOP, and COP-noCFP.

3. COP-Miner runs faster and consumes less memory than COP-efo. For example, on SDB6, from Fig. 3, COP-efo requires 0.497s, while COP-Miner only needs 0.349s. In Fig. 4, COP-efo occupies 21.822Mb, while COP-Miner occupies only 8.665Mb. The reason is as follows. COP-efo employs EFO-Miner to calculate the occurrences

of prefix pattern q and its fusion patterns, which requires pattern fusion and multiple support calculations. Thus, COP-efo has to generate more occurrences of superpatterns. Fig. 6 also validates this claim. On SDB6, COP-efo makes 101,420 comparisons, while COP-Miner makes only 609. Moreover, from Fig. 5, COP-Miner and COP-efo each generate 11 candidate patterns, since they all apply the CSS algorithm to calculate the supports of superpatterns. The same effect can be found on all other datasets. Hence, the method of preparation stage is more efficient than EFO-Miner in calculating the occurrences of pattern q and its fusion patterns, and COP-Miner outperforms COP-efo.

4. COP-Miner runs faster and requires less memory than both COP-enum and COP-noending. For example, on SDB8, from Fig. 3, COP-enum and COP-noending require 0.492s and 0.478s of running time, respectively, while COP-Miner needs 0.464s. In Fig. 4, COP-enum and COP-noending occupy 10.762MB and 9.865MB of running memory, respectively, while COP-Miner occupies only 9.461Mb. The reasons are as follows. The enumeration method generates more candidate patterns than the pattern fusion strategy. Fig. 5 also validates this claim. On SDB8, COP-enum checks 1,039 candidate patterns, while COP-Miner only checks 97. The ending strategy can further reduce the number of occurrences of superpatterns. Fig. 6 also validates this claim. On SDB8, COP-noending makes 3,345 comparisons, while COP-Miner makes 3,298. The same effect can be found on all other datasets. Therefore, COP-Miner outperforms COP-enum and COP-noending.

5. COP-Miner runs faster and consumes less memory than FAV-sliding. For example, on SDB2, it can be seen from Fig. 3 that FAV-sliding requires 0.297s, while COP-Miner needs 0.230s. In Fig. 4, FAV-sliding occupies 11.886Mb, while COP-Miner occupies 10.189Mb. The reason is as follows. We know that FAV-sliding adopts sliding windows to calculate the occurrences of all possible superpatterns. This method can be seen as a linear search to find the co-occurrence patterns. Thus, FAV-sliding has to check more candidate patterns. Fig. 5 also validates this claim. On SDB2, FAV-sliding checks 28 candidate patterns, while COP-Miner only checks 19. The same effect can be found on all other datasets. Hence, COP-Miner outperforms FAV-Sliding.

In summary, the experimental results verify that COP-Miner yields better performance than the other nine competing algorithms.

5.4 Scalability

To evaluate the scalability of COP-Miner, we selected SDB8 with length 60K as the experimental dataset. Moreover, we created 120K, 180K, 240K, 300K, 360K, 420K, and 480K, which were two, three, four, five, six, seven, and eight times the size of SDB8, respectively. We set $minsup=30$ and the prefix pattern was (1,3,2,5,4,6). Comparisons of running time and memory usage are shown in Figs. 7 and 8, respectively.

The results give rise to the following observations.

From Figs. 7 and 8, we know that both the running time and memory usage of COP-Miner show linear growth with the increase of the size of the dataset. For example, the size of 480K is eight times of SDB8. COP-Miner takes 0.422s on 60K, and 1.008s on 480K, giving $1.008/0.422=2.389$. The memory usage of COP-Miner is 19.059MB on 60K, and 81.521MB on 480K, giving $81.521/19.059=4.277$. Thus, the running time and memory usage grow significantly slower than the increase in the dataset size. These results indicate that the time and space complexities of COP-Miner are positively related to the dataset size. This phenomenon can be found on all other datasets. More importantly, we can see that COP-Miner is about two times faster than COP-efo, and the memory usage of COP-efo also exceeds COP-Miner. Therefore, we conclude that COP-Miner has strong scalability, since the mining performance does not degrade as the dataset size increases.

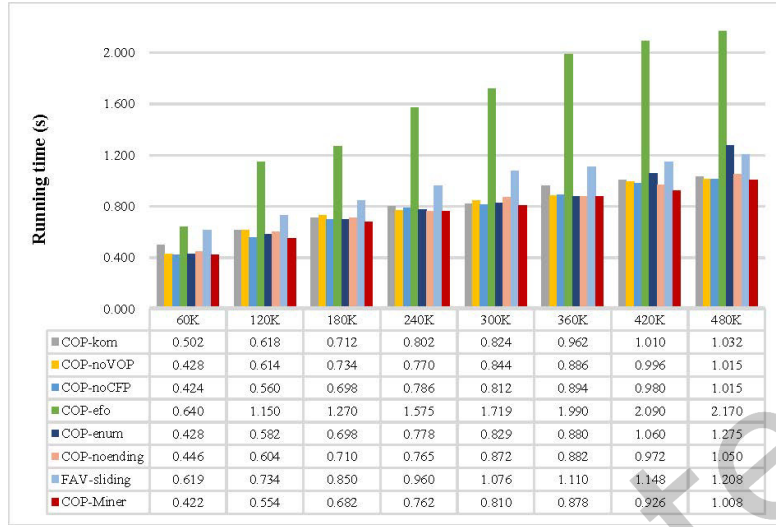


Fig. 7. Comparison of running time with different dataset sizes

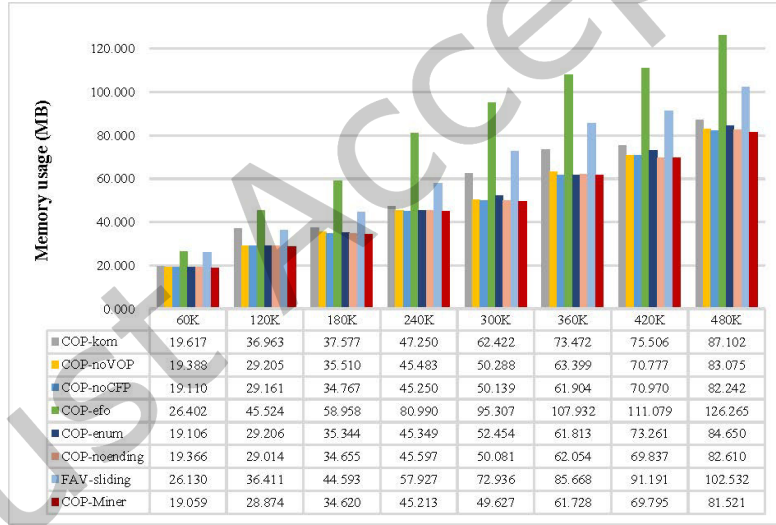


Fig. 8. Comparison of memory usage with different dataset sizes

5.5 Influence of parameters

We evaluated the effects of prefix patterns with different lengths and *minsup* on running time. To examine the influence, we selected SDB8 as the experimental dataset, which is the largest dataset in Table 3, and selected COP-kom, COP-noVOP, COP-noCFP, COP-efo, COP-enum, and COP-noending as the competing algorithms.

1) Influence of prefix patterns with different lengths:

To report the influence of prefix patterns with different lengths on running time, we set $minsup=12$. The prefix patterns on SDB8 are (1,3,2), (1,3,2,4), (1,3,2,5,4), (1,3,2,5,4,6), and (1,4,2,6,5,7,3) with lengths 3, 4, 5, 6, and 7, respectively, and all the seven algorithms discover 2121, 1059, 94, 64, and 36 COPs for the five prefix patterns with different lengths, respectively. The comparisons of running time and numbers of candidate patterns on SDB8 are shown in Figs. 9 and 10, respectively.

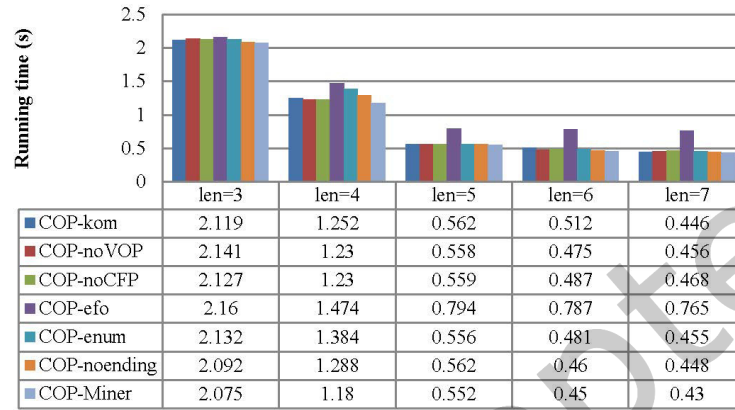


Fig. 9. Comparison of running time of prefix patterns with different lengths on SDB8

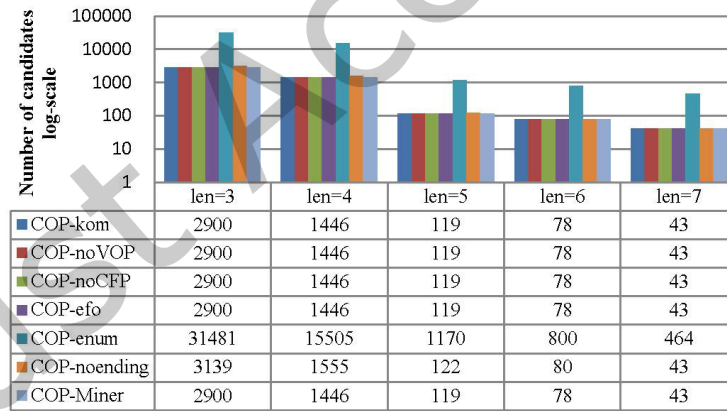


Fig. 10. Comparison of numbers of candidate patterns of prefix patterns with different lengths on SDB8

The results give rise to the following observations.

As the length of the prefix pattern increases, the running time decreases, since the numbers of COPs and candidate patterns decrease. For example, from Figs. 9 and 10, when $len=3$, COP-Miner takes 2.075s, checks 2900 candidate patterns, and discovers 2121 COPs, whereas when $len=7$, COP-Miner takes 0.430s, checks 43 candidate patterns, and discovers 36 COPs. This phenomenon can be found on all other algorithms. The reason is as follows. As the length of the prefix pattern increases, fewer patterns can be found, thereby reducing the number

of candidate patterns generated. We know that the fewer the candidate patterns, the shorter the running time. Thus, the running time also decreases. More importantly, COP-Miner outperforms other competing algorithms for any length prefix pattern, since it is the fastest algorithm, as explained in the analysis in Section 5.3.

2) Influence of different *minsup*:

To report the influence of different *minsup* on running time, we set *minsup*=8, 10, 12, 14, 16. The prefix pattern on SDB8 is (1,3,2,5,4,6). The seven algorithms discover 97, 82, 64, 48, and 39 COPs for five different *minsup*, respectively. The comparisons of running time and numbers of candidate patterns on SDB8 are shown in Figs. 11 and 12, respectively.

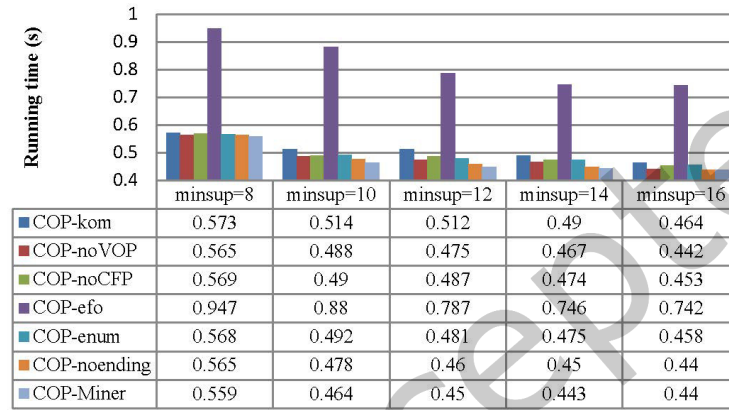


Fig. 11. Comparison of running time with different *minsup* on SDB8

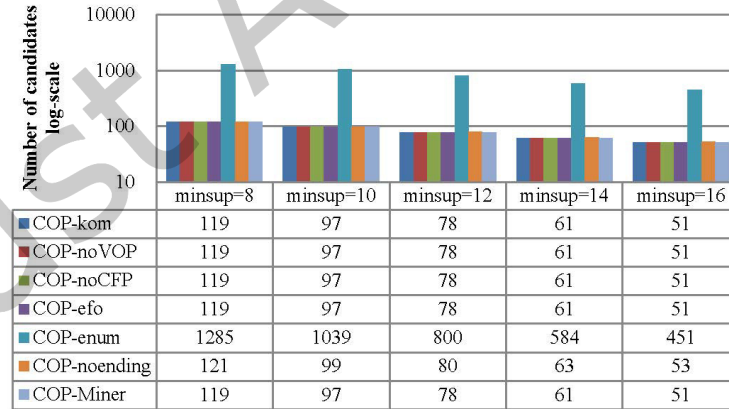


Fig. 12. Comparison of numbers of candidate patterns with different *minsup* on SDB8

The results give rise to the following observations.

As the *minsup* increases, the running time decreases, since the numbers of candidate patterns and COPs decrease. For example, from Figs. 11 and 12, when *minsup*=8, COP-Miner takes 0.559s, checks 119 candidate

patterns, and discovers 97 COPs, whereas when $minsup=16$, COP-Miner takes 0.440s, checks 51 candidate patterns, and discovers 39 COPs. This phenomenon can be found on all other algorithms. The reason for this is as follows. We know that for a certain prefix pattern, fewer patterns can be frequent patterns and candidate patterns as $minsup$ increases. Hence, fewer COPs can be found and the running time also decreases. Moreover, COP-Miner outperforms other competing algorithms for any $minsup$, since it is the fastest algorithm, as explained in the analysis in Section 5.3.

5.6 Case study

In this section, we predict intervals of future values based on trend patterns found in historical time series by using COP mining. Fig. 13 gives the specific prediction by using COPs obtained on the training set of SDB6. The left side of the vertical line is the prefix pattern $\mathbf{p} = (1,3,2,4)$ obtained from the historical data. The right side of the vertical line is based on the mined COPs to predict the trend that may occur in the next stage. We selected the two patterns with the highest frequency and the second highest frequency on the training set to predict the possible intervals at the sixth time point. The two dotted lines represent the possible interval, and the results predicted that the sixth hour should be warmer than 29.3 degrees Celsius. The solid line represents the actual trend and the actual temperature at that time was 29.8.

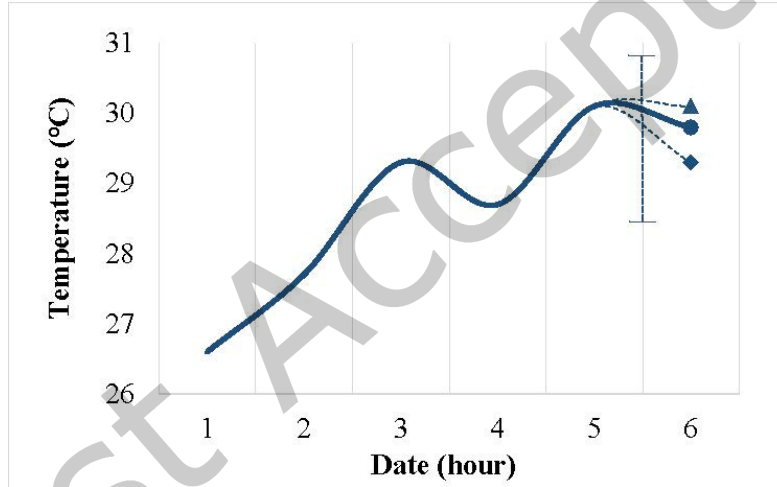


Fig. 13. COPs of pattern $\mathbf{p} = (1, 3, 2, 4)$ on the training set of SDB6

To compare the prediction performance of COP-Miner, we selected two commonly used models: ARIMA [27] and DTW [19] as comparison models, which predict the OPPs of subsequences similar to the prefix pattern on the original time series. Moreover, we also employed COP-noLEP as a competing algorithm to generate COPs, which does not use the LEP algorithm to eliminate the distortion of time series. We selected SDB1-SDB9 to evaluate the prediction performance and divided each dataset into a training set and a test set in a ratio of 8:2. The prefix patterns used for training are shown in Table 4.

To evaluate the performance of prediction based on the co-occurrence patterns, there are three commonly used criteria: precision $Pr = \frac{TP}{TP+FP}$, recall $Re = \frac{TP}{TP+FN}$, and F1-score $F1 = \frac{2 \times Pr \times Re}{Pr+Re}$, where TP , FP , and FN are the numbers of the correctly predicted patterns, wrongly predicted patterns, and unpredicted patterns in the test set, respectively. There is no wrongly predicted pattern, i.e. $FP=0$, which means that the frequent co-occurrence patterns mined in the training set all appear in the test set of these nine datasets. Thus, $Pr=1$. Therefore, we did

Table 4. Prefix patterns

Dataset	Prefix pattern	Dataset	Prefix pattern	Dataset	Prefix pattern
SDB1	(1,3,2,4)	SDB2	(1,4,2,3)	SDB3	(3,2,4,1)
SDB4	(3,1,5,2,4)	SDB5	(3,1,2,4,5)	SDB6	(1,4,2,3)
SDB7	(1,3,2)	SDB8	(3,2,5,1,4)	SDB9	(2,4,3,6,5,8,1,7)

not choose precision as the criterion, but rather chose recall and F1-score. Recall can analyze the weight of the co-occurrence patterns mined through the training set in the test set. F1-score is a comprehensive consideration of precision and recall, which can more comprehensively evaluate the accuracy of the prediction results. Taking SDB8 as an example, we selected the top two COPs in the training set for COP-Miner: (4,3,6,1,5,2), (4,3,6,2,5,1). This result means that the value of the sixth point should be smaller than the corresponding true value of rank 3. In the test set, all length-6 patterns with the prefix pattern (3,2,5,1,4) are (3,2,6,1,5,4), (4,3,6,1,5,2), (4,2,6,1,5,3), and (4,3,6,2,5,1), and their supports are 20, 118, 54, and 132, respectively. The sum of supports of patterns (4,3,6,1,5,2), (4,3,6,2,5,1) and total supports are 118+132=250 and 324, respectively. Thus, recall and F1-score of COP-Miner on the test set are $Re = \frac{250}{324} = 0.7716$ and $F1 = \frac{2 \times 0.7716}{1 + 0.7716} = 0.8711$. Comparison of recall and F1-score results are shown in Figs. 14 and 15, respectively.

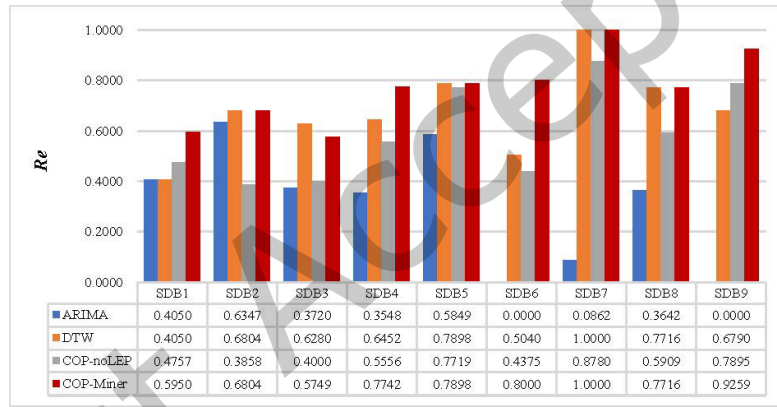


Fig. 14. Comparison of recall

The results give rise to the following observations.

COP-Miner has higher accuracy than ARIMA and DTW prediction models. For example, on SDB1, the recall and F1-score of COP-Miner are 0.5950 and 0.7461, respectively, while those of ARIMA and DTW both are 0.4050 and 0.5765, respectively. This phenomenon can be found on most other datasets. The reason is as follows. The prediction results of ARIMA and DTW cannot accurately describe the trend characteristics, while COP-Miner is closer to the real data as a whole. Therefore, COP-Miner has better prediction accuracy than ARIMA and DTW.

COP-Miner has higher accuracy than COP-noLEP. For example, on SDB4, the recall and F1-score of COP-Miner are 0.7742 and 0.8727, respectively, while those of COP-noLEP are 0.5556 and 0.7143. This phenomenon can be found on all other datasets. The reason for this is as follows. The difference between COP-Miner and COP-noLEP is that COP-Miner uses the LEP algorithm to extract keypoints, while COP-noLEP does not. The results indicate that LEP can improve accuracy by removing unimportant trend information in time series. Therefore, COP-Miner yields better prediction accuracy than COP-noLEP.

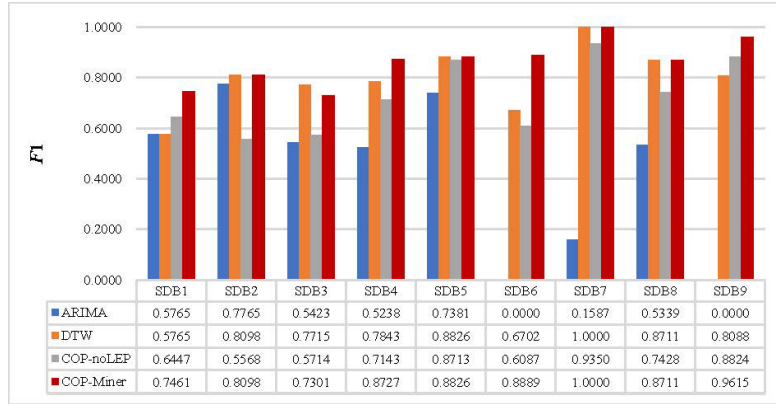


Fig. 15. Comparison of F1-score

6 CONCLUSION

To discover all COPs with the same prefix pattern efficiently, we propose the COP-Miner algorithm, which consists of three parts: 1) extracting keypoints, 2) preparation stage, and 3) iteratively calculating supports and mining frequent COPs. To reduce distortion interference in time series and avoid mining redundant patterns, we propose the LEP algorithm to extract local extreme points and to obtain keypoint sub-time series and keypoint time series, which can effectively improve the prediction performance compared to mining directly on the original time series. To prepare for the first round of mining, we propose the method of the preparation stage, which contains four steps: obtaining the suffix OPP of the keypoint sub-time series, calculating the occurrences of the suffix OPP, verifying the occurrences of the keypoint sub-time series, and calculating the occurrences of all fusion patterns of keypoint sub-time series. To calculate the support effectively, we propose the CSS algorithm, which adopts a pattern fusion method to generate candidate patterns and an ending strategy to further prune redundant patterns. Experimental results from nine real datasets verified that COP-Miner yields better running performance and scalability than other ten competing algorithms, especially on large-scale datasets, COP-Miner is about 40 times and 4 times faster than OPP-Miner and EFO-Miner, respectively. More importantly, COPs with keypoint alignment yield better prediction performance.

ACKNOWLEDGEMENT

This work was supported by Hebei Social Science Foundation Project (No. HB19GL055).

REFERENCES

- [1] Daichi Amagata and Takahiro Hara. 2019. Mining top-k co-occurrence patterns across multiple streams. *IEEE Transactions on Knowledge and Data Engineering* 29, 10 (2019), 2249–2262.
- [2] Xiang Ao, Haoran Shi, Jin Wang, Luo Zuo, Hongwei Li, and Qing He. 2019. Large-scale frequent episode mining from complex event sequences with hierarchies. *ACM Transactions on Intelligent Systems and Technology* 10, 4 (2019), 36.
- [3] Bing Bai, Guiling Li, Senzhang Wang, Zongda Wu, and Wenhe Yan. 2021. Time series classification based on multi-feature dictionary representation and ensemble learning. *Expert Systems with Applications* 169 (2021), 114162.
- [4] G. E. P. Box and David A. Pierce. 1970. Distribution of residual autocorrelations in autoregressive-integrated moving average time series models. *J. Amer. Statist. Assoc.* 65, 332 (1970), 1509–1526.
- [5] Kaushik Chakrabarti, Eamonn Keogh, Sharad Mehrotra, and Michael Pazzani. 2002. Locally adaptive dimensionality reduction for indexing large time series databases. *ACM Transactions on Database Systems* 27, 2 (2002), 188–228.

- [6] Shuhui Cheng, Youxi Wu, Yan Li, Fang Yao, and Fan Min. 2021. TWD-SFNN: Three-way decisions with a single hidden layer feedforward neural network. *Information Sciences* 579 (2021), 15–32.
- [7] Chenglong Dai, Jia Wu, Dechang Pi, Stefanie I. Becker, Lin Cui, Qin Zhang, and Blake Johnson. 2022. Brain EEG time-series clustering using maximum-weight clique. *IEEE Transactions on Cybernetics* 52, 1 (2022), 357–371.
- [8] Xiangjun Dong, Ping Qiu, Jinhu Lü, Longbing Cao, and Tiantian Xu. 2019. Mining top-k useful negative sequential patterns via learning. *IEEE Transactions on Neural Networks and Learning Systems* 30, 9 (2019), 2764–2778.
- [9] Philippe Fournier-Viger, Wensheng Gan, Youxi Wu, Mourad Nouioua, Wei Song, Tin C. Truong, and Hai Van Duong. 2022. Pattern mining: Current challenges and opportunities. *DASFAA 2022: Database Systems for Advanced Applications* (2022), 34–49.
- [10] Wensheng Gan, Jerry Chun-Wei Lin, Philippe Fournier-Viger, Han-Chieh Chao, and Philip S. Yu. 2019. A survey of parallel sequential pattern mining. *ACM Transactions on Knowledge Discovery from Data* 13, 3 (2019), 25.
- [11] Chao Gao, Lei Duan, Guozhu Dong, Haiqing Zhang, Hao Yang, and Changjie Tang. 2016. Mining top-k distinguishing sequential patterns with flexible gap constraints. In *International Conference on Web-Age Information Management*. Springer, 82–94.
- [12] Shaghayegh Gharghabi, Shima Imani, Anthony Bagnall, Amirali Darvishzadeh, and Eamonn Keogh. 2018. Matrix profile XII: MPdist: A novel time series distance measure to allow data mining in more challenging scenarios. In *IEEE International Conference on Data Mining*. IEEE, 965–970.
- [13] Shaghayegh Gharghabi, Shima Imani, Anthony Bagnall, Amirali Darvishzadeh, and Eamonn Keogh. 2020. An ultra-fast time series distance measure to allow data mining in more complex real-world deployments. *Data Mining and Knowledge Discovery* 34, 4 (2020), 1104–1135.
- [14] Dan Guo, Xuegang Hu, Fei Xie, and Xindong Wu. 2013. Pattern matching with wildcards and gap-length constraints based on a centrality-degree graph. *Applied Intelligence* 39, 1 (2013), 57–74.
- [15] Dan Guo, Ermao Yuan, Xuegang Hu, and Xindong Wu. 2018. Co-occurrence pattern mining based on a biological approximation scoring matrix. *Pattern Analysis and Applications* 21, 4 (2018), 977–996.
- [16] Nan Han, Shaojie Qiao, Kun Yue, Jianbin Huang, Tingting Tang Qiang He, Faliang Huang, Chunlin He, and Chang-An Yuan. 2022. Algorithms for trajectory points clustering in location-based social networks. *ACM Transactions on Intelligent Systems and Technology* 13, 3 (2022), 43.
- [17] Zengyou He, Ziyao Wu, Guangyao Xu, Yan Liu, and Quan Zou. 2023. Decision tree for sequences. *IEEE Transactions on Knowledge and Data Engineering* 35, 1 (2023), 251–263.
- [18] Gengsen Huang, Wensheng Gan, Jian Weng, and Philip S. Yu. 2023. US-rule: Discovering utility-driven sequential rules. *ACM Transactions on Knowledge Discovery from Data* 17, 1 (2023), 10.
- [19] Eamonn Keogh and Chotirat Ann Ratanamahatana. 2005. Exact indexing of dynamic time warping. *Knowledge and Information Systems* 7 (2005), 358–386.
- [20] Jinil Kim, Peter Eades, Rudolf Fleischer, Seok-Hee Hong, Costas S. Iliopoulos, Kunsoo Park, Simon J. Puglisi, and Takeshi Tokuyama. 2014. Order-preserving matching. *Theoretical Computer Science* 525 (2014), 68–79.
- [21] Youngho Kim, Munseong Kang, Joong Chae Na, and Jeong Seop Sim. 2023. Order-preserving pattern matching with scaling. *Inform. Process. Lett.* 180 (2023), 106333.
- [22] Abdelmajid Lahreche and Bachir Boucheham. 2021. A fast and accurate similarity measure for long time series classification based on local extrema and dynamic time warping. *Expert Systems with Applications* 168 (2021), 114374.
- [23] Wu Lee, Yuliang Shi, Hongfeng Sun, Lin Cheng, Kun Zhang, Xinjun Wang, and Zhiyong Chen. 2022. MSIPA: Multi-scale interval pattern-aware network for ICU transfer prediction. *ACM Transactions on Knowledge Discovery from Data* 16, 1 (2022), 1–17.
- [24] Chun Li, Qingyan Yang, Jianyong Wang, and Ming Li. 2012. Efficient mining of gap-constrained subsequences and its various applications. *ACM Transactions on Knowledge Discovery from Data* 6, 1 (2012), 2.
- [25] Qing Li, Jinghua Tan, Jun Wang, and Hsinchun Chen. 2021. A multimodal event-driven LSTM model for stock prediction using online news. *IEEE Transactions on Knowledge and Data Engineering* 33, 10 (2021), 3323–3337.
- [26] Qingzhe Li, Liang Zhao, Yi-Ching Lee, and Jessica Lin. 2020. Contrast pattern mining in paired multivariate time series of a controlled driving behavior experiment. *ACM Transactions on Spatial Algorithms and Systems* 6, 4 (2020), 25.
- [27] Yangfan Li, Kenli Li, Cen Chen, Xu Zhou, Zeng Zeng, and Keqin Li. 2022. Modeling temporal patterns with dilated convolutions for time-series forecasting. *ACM Transactions on Knowledge Discovery from Data* 16, 1 (2022), 1–22.
- [28] Yan Li, Lei Yu, Jing Liu, Lei Guo, Youxi Wu, and Xindong Wu. 2022. NetDPO: (delta, gamma)-approximate pattern matching with gap constraints under one-off condition. *Applied Intelligence* 52, 11 (2022), 12155–12174.
- [29] Yan Li, Chang Zhang, Jie Li, Wei Song, Zhenlian Qi, Youxi Wu, and Xindong Wu. 2023. MCoR-Miner: Maximal co-occurrence nonoverlapping sequential rule mining. *IEEE Transactions on Knowledge and Data Engineering* 35, 9 (2023), 9531–9546.
- [30] Yan Li, Shuai Zhang, Lei Guo, Jing Liu, Youxi Wu, and Xindong Wu. 2022. NetNMSP: Nonoverlapping maximal sequential pattern mining. *Applied Intelligence* 52, 9 (2022), 9861–9884.
- [31] Zhenhui Li, Jiawei Han, Ming Ji, Lu-An Tang, Yintao Yu, Bolin Ding, Jae-Gil Lee, and Roland Kays. 2011. MoveMine: Mining moving object data for discovery of animal movement patterns. *ACM Transactions on Intelligent Systems and Technology* 2, 4 (2011), 37.

- [32] Jessica Lin, Eamonn Keogh, Li Wei, and Stefano Lonardi. 2007. Experiencing SAX: A novel symbolic representation of time series. *Data Mining and Knowledge Discovery* 15 (2007), 107–144.
- [33] Qi Lin, Wensheng Gan, Yongdong Wu, Jiahui Chen, and Chien-Ming Chen. 2022. Smart system: Joint utility and frequency for pattern classification. *ACM Transactions on Management Information Systems* 13, 4 (2022), 43.
- [34] Kunpeng Liu, Yanjie Fu, Le Wu, Xiaolin Li, Charu Aggarwal, and Hui Xiong. 2023. Automated feature selection: A reinforcement learning perspective. *IEEE Transactions on Knowledge and Data Engineering* 35, 3 (2023), 2272–2284.
- [35] Pengfei Ma, Youxi Wu, Yan Li, Lei Guo, He Jiang, Xingquan Zhu, and Xindong Wu. 2022. HW-Forest: Deep forest with hashing screening and window screening. *ACM Transactions on Knowledge Discovery from Data* 16, 6 (2022), 123.
- [36] Ioannis Mavroudopoulos and Anastasios Gounaris. 2023. SIESTA: A scalable infrastructure of sequential pattern analysis. *IEEE Transactions on Big Data* 9, 3 (2023), 975–990.
- [37] Fan Min, Zhi-Heng Zhang, Wenjie Zhai, and Rong-Ping Shen. 2020. Frequent pattern discovery with tri-partition alphabets. *Information Sciences* 507 (2020), 715–732.
- [38] Saqib Nawaz, Philippe Fournier-Viger, M. Zohaib Nawaz, Guoting Chen, and Youxi Wu. 2022. MalSPM: Metamorphic malware behavior analysis and classification using sequential pattern mining. *Computers & Security* 118 (2022), 102741.
- [39] Saqib Nawaz, Philippe Fournier-Viger, Unil Yun, Youxi Wu, and Wei Song. 2022. Mining high utility itemsets with hill climbing and simulated annealing. *ACM Transactions on Management Information Systems* 13, 1 (2022), 4.
- [40] Filipe Rodrigues, Loulia Markou, and Francisco C. Pereira. 2019. Combining time-series and textual data for taxi demand prediction in event areas: A deep learning approach. *Information Fusion* 49 (2019), 120–129.
- [41] Marco Storace and Oscar De Feo. 2004. Piecewise-linear approximation of nonlinear dynamical systems. *IEEE Transactions on Circuits and Systems I: Regular Papers* 51, 4 (2004), 830–842.
- [42] Tin Truong, Hai Duong, B Le, and Philippe Fournier-Viger. 2019. FMaxCloHUSM: An efficient algorithm for mining frequent closed and maximal high utility sequences. *Engineering Applications of Artificial Intelligence* 85 (2019), 1–20.
- [43] Bay Vo, Sang Pham, Tuong Le, and Zhi-Hong Deng. 2017. A novel approach for mining maximal frequent patterns. *Expert Systems with Applications* 73 (2017), 178–186.
- [44] Lizhen Wang, Xuguang Bao, and Lihua Zhou. 2018. Redundancy reduction for prevalent co-location patterns. *IEEE Transactions on Knowledge and Data Engineering* 30, 1 (2018), 142–155.
- [45] Lizhen Wang, Yuan Fang, and Lihua Zhou. 2022. Preference-based spatial co-location pattern mining. *Springer Singapore* (2022). <https://doi.org/10.1007/978-981-16-7566-9>
- [46] Wei Wang, Jing Tian, Fang Lv, Guodong Xin, Yingfan Ma, and Bailing Wang. 2021. Mining frequent pyramid patterns from time series transaction data with custom constraints. *Computers & Security* 100 (2021), 102088.
- [47] Yuehua Wang, Youxi Wu, Yan Li, Fang Yao, Philippe Fournier-Viger, and Xindong Wu. 2022. Self-adaptive nonoverlapping sequential pattern mining. *Applied Intelligence* 52, 6 (2022), 6646–6661.
- [48] Xindong Wu, Xingquan Zhu, and Minghui Wu. 2022. The evolution of search: Three computing paradigms. *ACM Transactions on Management Information Systems* 13, 2 (2022), 20.
- [49] Youxi Wu, Mingjie Chen, Yan Li, Jing Liu, Zhao Li, Jinyan Li, and Xindong Wu. 2023. ONP-Miner: One-off negative sequential pattern mining. *ACM Transactions on Knowledge Discovery from Data* 17, 3 (2023), 37.
- [50] Youxi Wu, Shuhui Cheng, Yan Li, Rongjie Lv, and Fan Min. 2023. STWD-SFNN: Sequential three-way decisions with a single hidden layer feedforward neural network. *Information Sciences* 632 (2023), 299–323.
- [51] Youxi Wu, Meng Geng, Yan Li, Lei Guo, Zhao Li, Philippe Fournier-Viger, Xingquan Zhu, and Xindong Wu. 2021. HANP-Miner: High average utility nonoverlapping sequential pattern mining. *Knowledge-Based Systems* 229 (2021), 107361.
- [52] Youxi Wu, Qian Hu, Yan Li, Lei Guo, Xingquan Zhu, and Xindong Wu. 2023. OPP-Miner: Order-preserving sequential pattern mining for time series. *IEEE Transactions on Cybernetics* 53, 5 (2023), 3288–3300.
- [53] Youxi Wu, Lanfang Luo, Yan Li, Lei Guo, Philippe Fournier-Viger, Xingquan Zhu, and Xindong Wu. 2022. NTP-Miner: Nonoverlapping three-way sequential pattern mining. *ACM Transactions on Knowledge Discovery from Data* 16, 3 (2022), 51.
- [54] Youxi Wu, Yufei Meng, Yan Li, Lei Guo, Xingquan Zhu, Philippe Fournier-Viger, and Xindong Wu. 2023. COPP-Miner: Top-k contrast order-preserving pattern mining for time series classification. *IEEE Transactions on Knowledge and Data Engineering* (2023). <https://doi.org/10.1109/TKDE.2023.3321749>
- [55] Youxi Wu, Yao Tong, Xingquan Zhu, and Xindong Wu. 2018. NOSEP: Nonoverlapping sequence pattern mining with gap constraints. *IEEE Transactions on Cybernetics* 48, 10 (2018), 2809–2822.
- [56] Youxi Wu, Lingling Wang, Jiadong Ren, Wei Ding, and Xindong Wu. 2014. Mining sequential patterns with periodic wildcard gaps. *Applied Intelligence* 41, 1 (2014), 99–116.
- [57] Youxi Wu, Xiaohui Wang, Yan Li, Lei Guo, Zhao Li, Ji Zhang, and Xindong Wu. 2022. OWSP-Miner: Self-adaptive one-off weak-gap strong pattern mining. *ACM Transactions on Management Information Systems* 13, 3 (2022), 25.
- [58] Youxi Wu, Yuehua Wang, Yan Li, Xingquan Zhu, and Xindong Wu. 2022. Top-k self-adaptive contrast sequential pattern mining. *IEEE Transactions on Cybernetics* 52, 11 (2022), 11819–11833.

- [59] Youxi Wu, Xiaoqian Zhao, Yan Li, Lei Guo, Xingquan Zhu, Philippe Fournier-Viger, and Xindong Wu. 2023. OPR-Miner: Order-preserving rule mining for time series. *IEEE Transactions on Knowledge and Data Engineering* 35, 11 (2023), 11722–11735.
- [60] Youxi Wu, Changrui Zhu, Yan Li, Lei Guo, and Xindong Wu. 2020. NetNCSP: Nonoverlapping closed sequential pattern mining. *Knowledge-Based Systems* 196 (2020), 105812.
- [61] Fei Xie, Xindong Wu, and Xingquan Zhu. 2017. Efficient sequential pattern mining with wildcards for keyphrase extraction. *Knowledge-Based Systems* 115 (2017), 27–39.
- [62] Surong Yan, Kwei-Jay Lin, Xiaolin Zheng, and Wenyu Zhang. 2020. Using latent knowledge to improve real-time activity recognition for smart IoT. *IEEE Transactions on Knowledge and Data Engineering* 32, 3 (2020), 574–587.
- [63] Jinyoung Yeo, Seungwon Hwang, Sungchul Kim, Eunye Koh, and Nedim Lipka. 2020. Conversion prediction from clickstream: Modeling market prediction and customer predictability. *IEEE Transactions on Knowledge and Data Engineering* 32, 2 (2020), 246–259.
- [64] Jianming Zhan, Jin Ye, Weiping Ding, and Peide Liu. 2022. A novel three-way decision model based on utility theory in incomplete fuzzy decision systems. *IEEE Transactions on Fuzzy Systems* 30, 7 (2022), 2210–2226.
- [65] Shuainan Zhang, Yafeng He, Xuzhao Li, Wude Yang, and Ying Zhou. 2021. Biolabel-led research pattern positions the effects and mechanisms of *Sophorae Tonkinensis* radix et rhizome on lung diseases: A novel strategy for computer-aided herbal medicine research based on omics and bioinformatics. *Computers in Biology and Medicine* 136 (2021), 104769.
- [66] Branislav Đurian, Jan Holub, Hannu Peltola, and Jorma Tarhio. 2010. Improving practical exact string matching. *Inform. Process. Lett.* 110, 4 (2010), 148–152.